

# LAPORAN DOKUMENTASI PROYEK AI LEARNMATE

## M1. Project Setup and Initial Prompt Exploration

### 1.1 Team Formation and Project Planning

#### Objective

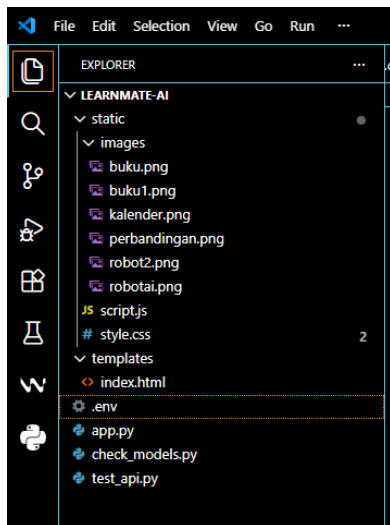
To define project goals, features, and development workflow.

#### Activities

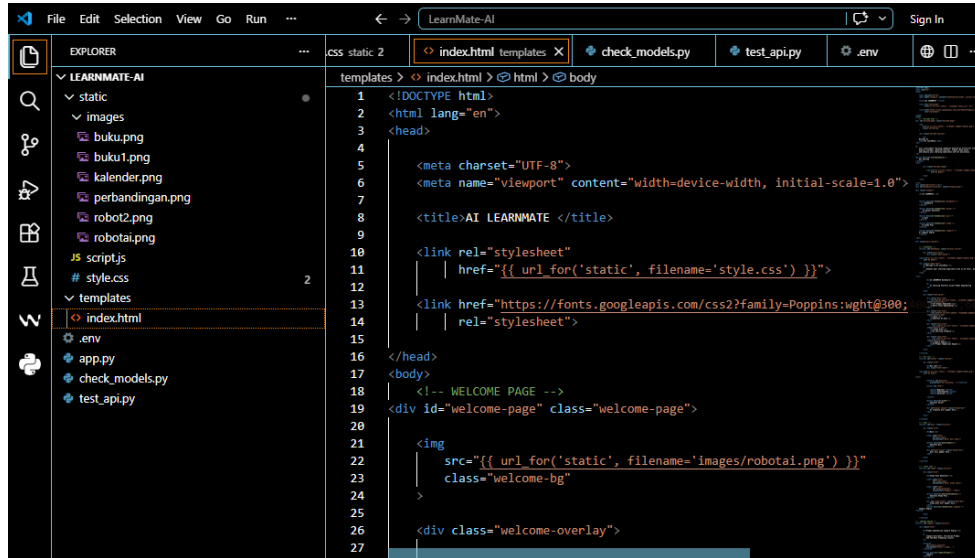
1. Form the project team.
2. Define project objectives.
3. Determine the main features:
  - AI Answer Generator
  - Multi-Level Explanation Generator
  - Study Plan Generator
  - Quiz Generator
  - Prompt Comparison Engine
4. Create project timeline and task distribution.

#### Screenshot Required

- Project planning document



- Team discussion or project board



## 1.2 LLM Selection and API Setup

## 1.2 LLM Selection and API Setup

### Objective

To select the Large Language Model and connect it with the application.

### Activities

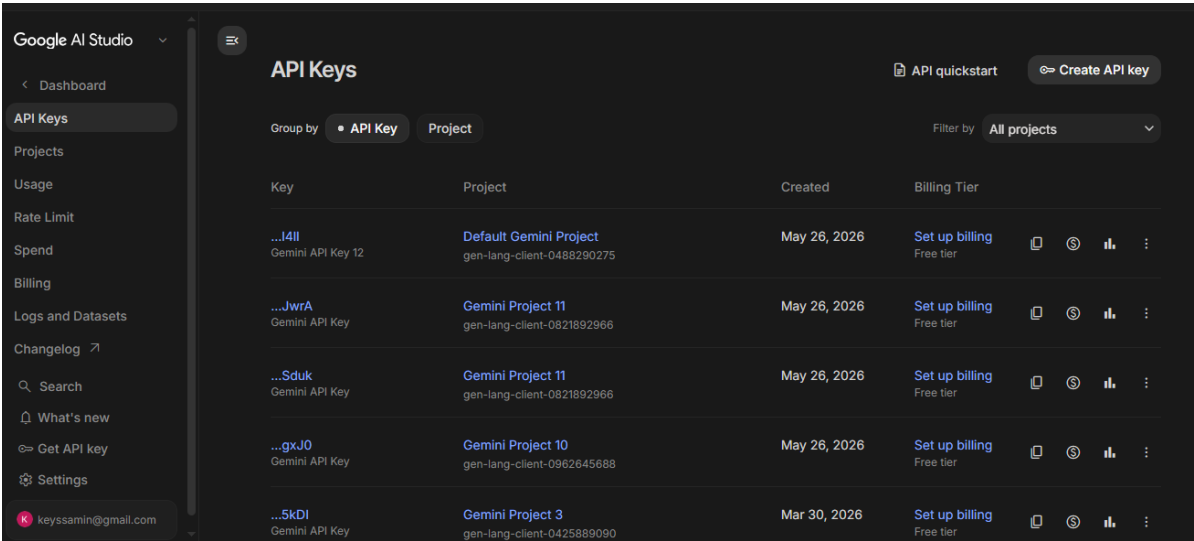
1. Compare available LLMs.
2. Select Google Gemini.
3. Create API Key from Google AI Studio.
4. Configure environment variables.
5. Connect Flask with Gemini API.

### Technologies Used

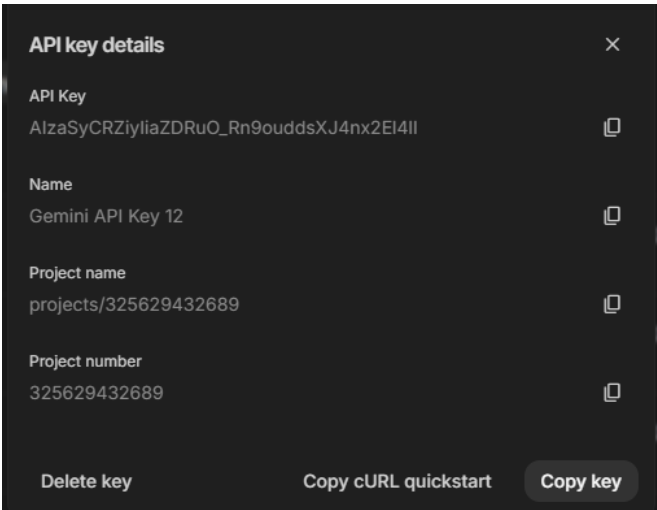
- Python
- Flask
- Gemini API
- HTML/CSS/JavaScript

Screenshot Required

- Google AI Studio API key page



- .env configuration



- Successful API connection

```
Command Prompt
3)
Requirement already satisfied: pyasn1-modules>=0.2.1 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from google-auth>=2.15.0->google-generativeai) (0.4.2)
Requirement already satisfied: cryptography>=38.0.3 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from google-auth>=2.15.0->google-generativeai) (48.0.0)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from requests<3.0.0,>=2.20.0->google-api-core->google-generativeai) (3.4.3)
Requirement already satisfied: idna<4,>=2.5 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from requests<3.0.0,>=2.20.0->google-api-core->google-generativeai) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from requests<3.0.0,>=2.20.0->google-api-core->google-generativeai) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from requests<3.0.0,>=2.20.0->google-api-core->google-generativeai) (2025.8.3)
Requirement already satisfied: colorama in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from click>=8.1.3->flask) (0.4.6)
Requirement already satisfied: cffi>=2.0.0 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from cryptography>=38.0.3->google-auth>=2.15.0->google-generativeai) (2.0.0)
Requirement already satisfied: pycparser in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from cffi>=2.0.0->cryptography>=38.0.3->google-auth>=2.15.0->google-generativeai) (2.23)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from pyasn1-modules>=0.2.1->google-auth>=2.15.0->google-generativeai) (0.6.3)
Requirement already satisfied: httplib2<1.0.0,>=0.19.0 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from google-api-python-client->google-generativeai) (0.31.2)
Requirement already satisfied: google-auth-httplib2<1.0.0,>=0.2.0 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from google-api-python-client->google-generativeai) (0.4.0)
Requirement already satisfied: uritemplate<5,>=3.0.1 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from google-api-python-client->google-generativeai) (4.2.0)
Requirement already satisfied: pyparsing<4,>=3.1 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from httplib2<1.0.0,>=0.19.0->google-api-python-client->google-generativeai) (3.2.1)
Requirement already satisfied: annotated-types>=0.6.0 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from pydantic->google-generativeai) (0.7.0)
Requirement already satisfied: pydantic-core==2.46.4 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from pydantic->google-generativeai) (2.46.4)
Requirement already satisfied: typing-inspection>=0.4.2 in c:\users\user\appdata\local\programs\python\python313\lib\site-packages (from pydantic->google-generativeai) (0.4.2)

[notice] A new release of pip is available: 25.3 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\user>
```

## 1.3 Basic Prompt Exploration

### Objective

To observe how different prompts affect AI responses.

### Activities

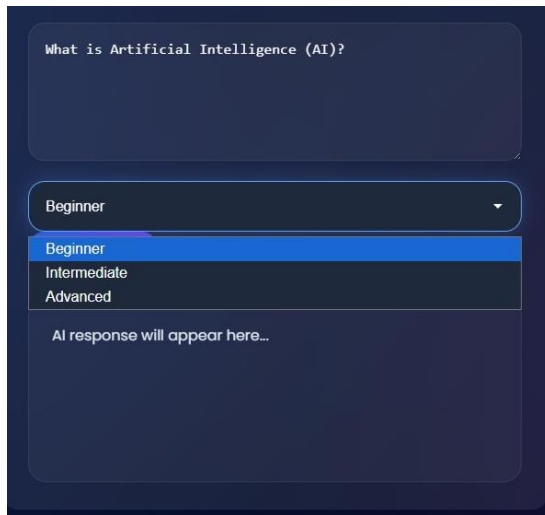
1. Test simple prompts.
2. Test detailed prompts.
3. Compare response quality.
4. Record observations.

### Example Prompt

What is Artificial Intelligence?

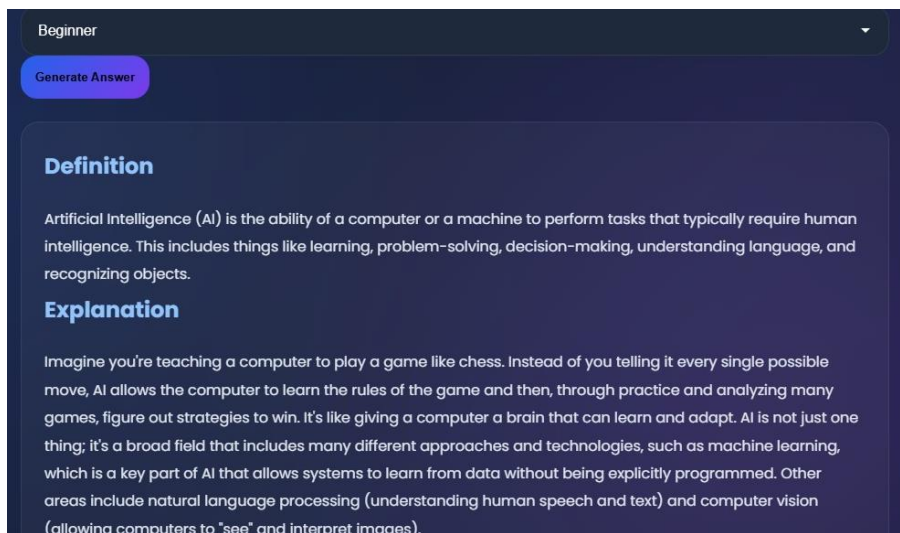
## Screenshot Required

- Prompt examples



A screenshot of a Gemini AI interface. At the top, a text input field contains the prompt "What is Artificial Intelligence (AI)?". Below the input field is a dropdown menu currently set to "Beginner". The dropdown menu is open, showing three options: "Beginner" (highlighted in blue), "Intermediate", and "Advanced". Below the dropdown menu, a large text area contains the placeholder text "AI response will appear here...".

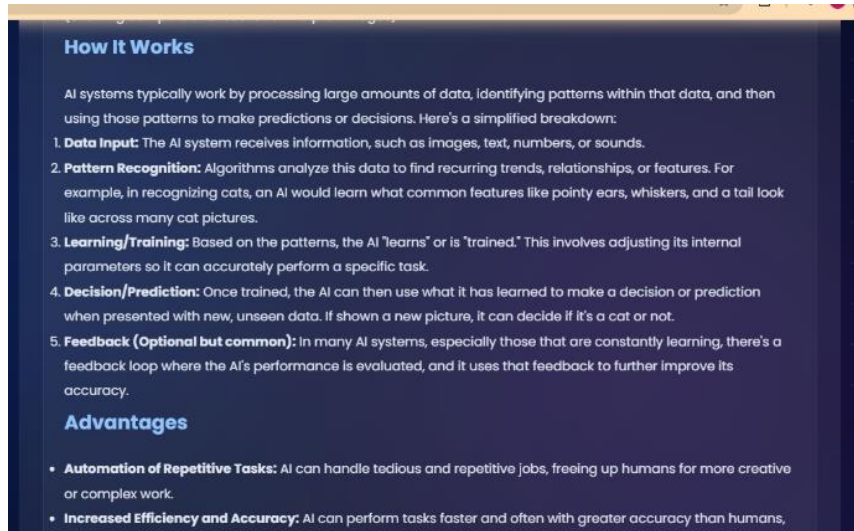
- Gemini responses



- Comparison results

```
•  
• prompt = f"""  
• You are AI LEARNMATE, a professional AI tutor and learning assistant.  
•  
• Your task is to explain the topic clearly and professionally.  
•
```

- Return ONLY valid HTML.
- 
- Required Structure:
- 
- `<h2>Definition</h2>`
- `<p>Provide a clear and concise definition.</p>`
- 
- `<h2>Explanation</h2>`
- `<p>Provide a detailed explanation based on the user's level.</p>`
- 
- `<h2>How It Works</h2>`
- `<p>Explain how the concept works step by step.</p>`
- 
- `<h2>Advantages</h2>`
- `<ul>`
- `<li>Advantage 1</li>`
- `<li>Advantage 2</li>`
- `<li>Advantage 3</li>`
- `</ul>`
- 
- `<h2>Example</h2>`
- `<p>Provide a real-world example that is easy to understand.</p>`
- 
- `<h2>Summary</h2>`
- `<ul>`
- `<li>Key point 1</li>`
- `<li>Key point 2</li>`
- `<li>Key point 3</li>`
- `</ul>`
- 
- Rules:
- - Use English only.
- - Return clean HTML only.
- - Do not use Markdown.
- - Do not use `##`, `###`, `**`, `*`, or ``` .
- - Make the explanation educational and beginner-friendly.
- - Adjust the complexity according to the user's level.
- - Use proper paragraphs and lists.
- - Keep the explanation organized and professional.
- 
- User Level:
- `{level}`
- 
- Topic:
- `{question}`



## 2.1 Structured Answer Generator Design

### Objective

Generate answers with consistent structure.

### Prompt Design

Definition  
Explanation  
How It Works  
Advantages  
Example  
Summary

### Implementation

Flask route:

- /ask

### Screenshot Required

- Prompt template

```
# =====  
# AI ANSWER GENERATOR
```

```
# =====
```

```
@app.route("/ask", methods=["POST"])
```

```
def ask():
```

```
    try:
```

```
        data = request.get_json()
```

```
        question = data.get("question")
```

```
        level = data.get("level")
```

```
        if not question:
```

```
            return jsonify({
```

```
                "answer": "Silakan masukkan pertanyaan terlebih dahulu."
```

```
            })
```

```
        prompt = f"""
```

```
You are AI LEARNMATE, a professional AI tutor and learning assistant.
```

```
Your task is to explain the topic clearly and professionally.
```

```
Return ONLY valid HTML.
```

```
Required Structure:
```

```
<h2>Definition</h2>
```

```
<p>Provide a clear and concise definition.</p>
```

```
<h2>Explanation</h2>
```

```
<p>Provide a detailed explanation based on the user's level.</p>
```

```
<h2>How It Works</h2>
```

```
<p>Explain how the concept works step by step.</p>
```

```
<h2>Advantages</h2>
```

```
<ul>
```

```
<li>Advantage 1</li>
```

```
<li>Advantage 2</li>
```

```
<li>Advantage 3</li>
```

```
</ul>
```

```
<h2>Example</h2>
```

```
<p>Provide a real-world example that is easy to understand.</p>
```

```
<h2>Summary</h2>
```



```
<ul>
<li>Key point 1</li>
<li>Key point 2</li>
<li>Key point 3</li>
</ul>
```

Rules:

- Use English only.
- Return clean HTML only.
- Do not use Markdown.
- Do not use ##, ###, \*\*, \*, or ```.
- Make the explanation educational and beginner-friendly.
- Adjust the complexity according to the user's level.
- Use proper paragraphs and lists.
- Keep the explanation organized and professional.

User Level:

```
{level}
```

Topic:

```
{question}
```

```
"""
```

```
    time.sleep(1)
```

```
    response = model.generate_content(prompt)
```

```
    return jsonify({
        "answer": response.text
    })
```

```
except Exception as e:
```

```
    return jsonify({
        "answer": f"Error: {str(e)}"
    })
```

```
# =====
```

```
# QUIZ GENERATOR
```

```
# =====
```

```
@app.route("/quiz", methods=["POST"])
```

```
def quiz():
```

```
    try:
```

```
data = request.get_json()

topic = data.get("topic")

if not topic:
    return jsonify({
        "quiz": "Silakan masukkan topik quiz."
    })

prompt = f"""
You are an expert educational quiz generator.
```

Generate 5 multiple-choice questions about:

{topic}

Return ONLY valid HTML.

Required Structure:

```
<h2>{topic} Quiz</h2>
```

```
<h3>Question 1</h3>
```

```
<p>Question text</p>
```

```
<ul>
```

```
<li>A. Option A</li>
```

```
<li>B. Option B</li>
```

```
<li>C. Option C</li>
```

```
<li>D. Option D</li>
```

```
</ul>
```

```
<p><strong>Correct Answer:</strong> A</p>
```

```
<p><strong>Explanation:</strong> Brief explanation.</p>
```

Repeat the same format until Question 5.

Rules:

- Use English.
- Create exactly 5 questions.
- Include 4 answer choices for each question.
- Include the correct answer and explanation.
- Questions should range from easy to challenging.

- Return clean HTML only.
- No Markdown.

"""

```
response = model.generate_content(prompt)
```

```
return jsonify({  
    "quiz": response.text  
})
```

```
except Exception as e:
```

```
return jsonify({  
    "quiz": f"Error: {str(e)}"  
})
```

```
# =====
```

```
# STUDY PLAN GENERATOR
```

```
# =====
```

```
@app.route("/study-plan", methods=["POST"])
```

```
def study_plan():
```

```
    try:
```

```
        data = request.get_json()
```

```
        topic = data.get("topic")
```

```
        duration = data.get("duration")
```

```
        if not topic or not duration:
```

```
            return jsonify({  
                "study_plan": "Topik dan durasi harus diisi."  
            })
```

```
        prompt = f"""
```

You are a professional AI Study Planner.

Create a detailed and structured study plan.

Topic:

{topic}

Duration:

{duration}

Return ONLY valid HTML.

Required Format:

```
<h2>Study Plan: {topic}</h2>
```

```
<h3>Day 1</h3>
```

```
<ul>
```

```
<li><strong>Topics:</strong> What to study</li>
```

```
<li><strong>Objectives:</strong> Learning goals</li>
```

```
<li><strong>Practice:</strong> Exercises or activities</li>
```

```
<li><strong>Estimated Time:</strong> Recommended study time</li>
```

```
</ul>
```

```
<h3>Day 2</h3>
```

```
<ul>
```

```
<li><strong>Topics:</strong> What to study</li>
```

```
<li><strong>Objectives:</strong> Learning goals</li>
```

```
<li><strong>Practice:</strong> Exercises or activities</li>
```

```
<li><strong>Estimated Time:</strong> Recommended study time</li>
```

```
</ul>
```

Continue until the specified duration is completed.

Rules:

- Use English only.
- Create a realistic and progressive learning schedule.
- Include clear objectives for each day.
- Include practical exercises or tasks.
- Return clean HTML only.
- Do not use Markdown.
- Do not use ##, ###, \*\*, \*, or code blocks.

```
"""
```

```
    response = model.generate_content(prompt)
```

```
    return jsonify({
        "study_plan": response.text
    })
```

```
except Exception as e:
```

```
    return jsonify({
        "study_plan": f"Error: {str(e)}"
```

```

    })
# =====
# COMPARE ENGINE
# =====
@app.route("/compare", methods=["POST"])
def compare():

    try:

        data = request.get_json()
        question = data.get("question")

        if not question:
            return jsonify({
                "error": "Please enter a topic."
            })

        # =====
        # ZERO SHOT PROMPT
        # =====
        zero_prompt = f"""
You are an AI tutor.

Explain the following topic:

{question}

Rules:
- Use English.
- Return clean HTML only.
- Include Definition, Explanation, Example, and Summary.
"""

        # =====
        # STRUCTURED PROMPT
        # =====
        structured_prompt = f"""
You are a professional tutor.

Explain the topic:

{question}

Use EXACTLY this HTML structure:

```

```
<h2>Definition</h2>
```

```
<p>Definition here.</p>
```

```
<h2>How It Works</h2>
```

```
<p>Explanation here.</p>
```

```
<h2>Advantages</h2>
```

```
<ul>
```

```
<li>Advantage 1</li>
```

```
<li>Advantage 2</li>
```

```
<li>Advantage 3</li>
```

```
</ul>
```

```
<h2>Example</h2>
```

```
<p>Example here.</p>
```

```
<h2>Summary</h2>
```

```
<ul>
```

```
<li>Point 1</li>
```

```
<li>Point 2</li>
```

```
<li>Point 3</li>
```

```
</ul>
```

Rules:

- Use English only.
- Return HTML only.
- No Markdown.

```
"""
```

```
# =====
```

```
# FEW SHOT PROMPT
```

```
# =====
```

```
fewshot_prompt = f"""
```

Example:

Topic: Internet

```
<h2>Definition</h2>
```

```
<p>
```

The Internet is a global network that connects computers and devices worldwide.

```
</p>
```

```
<h2>How It Works</h2>
```

```
<p>
```

Devices communicate through networks using internet protocols.

</p>

<h2>Advantages</h2>

<ul>

<li>Fast communication</li>

<li>Access to information</li>

<li>Online services</li>

</ul>

<h2>Example</h2>

<p>

Google, YouTube, Email, and Online Banking.

</p>

<h2>Summary</h2>

<ul>

<li>Connects people globally</li>

<li>Provides information</li>

<li>Supports digital services</li>

</ul>

Now explain:

{question}

Use the SAME format.

Rules:

- Use English only.

- Return HTML only.

- No Markdown.

"""

```
# =====
```

```
# GEMINI RESPONSES
```

```
# =====
```

```
zero_response = model.generate_content(  
    zero_prompt  
)
```

```
structured_response = model.generate_content(  
    structured_prompt  
)
```

```
fewshot_response = model.generate_content(  
    fewshot_prompt
```

```

        fewshot_prompt
    ).text

    return jsonify({
        "zero_shot": zero_response,
        "structured": structured_response,
        "few_shot": fewshot_response
    })

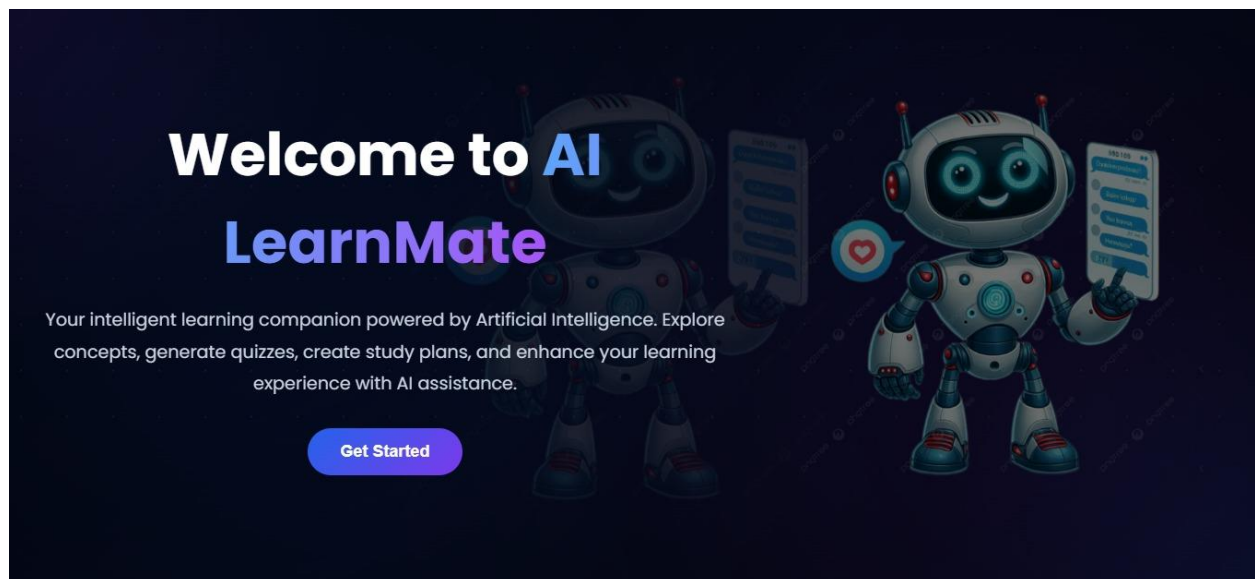
except Exception as e:

    return jsonify({
        "error": str(e)
    })

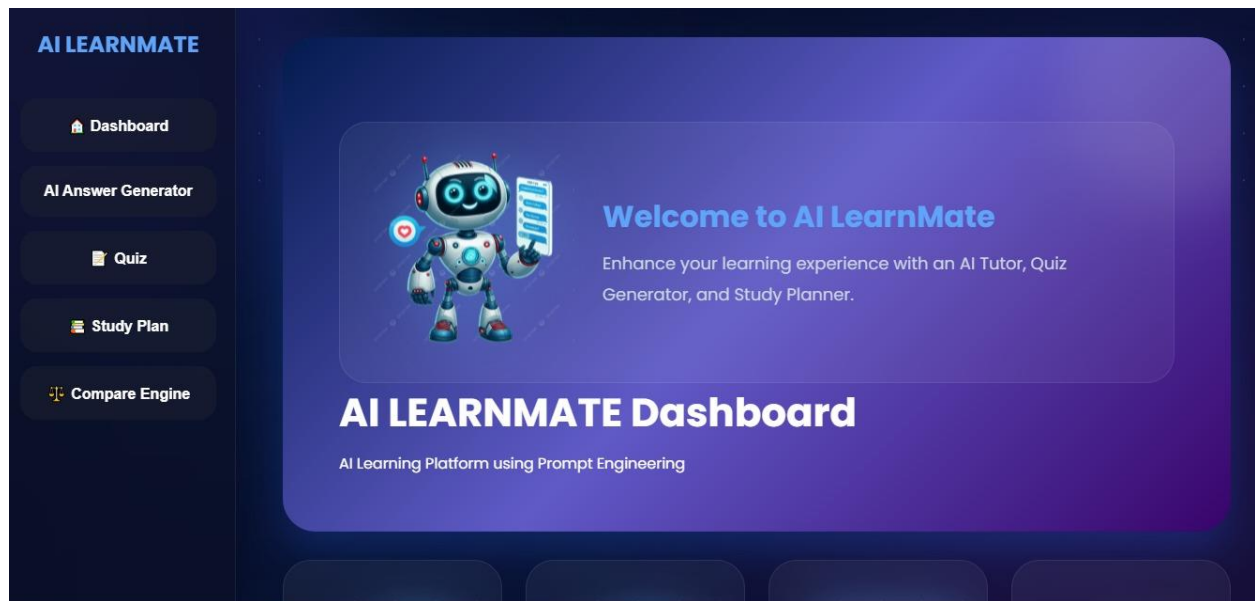
# =====
# RUN APP
# =====
if __name__ == "__main__":
    app.run(debug=True)

```

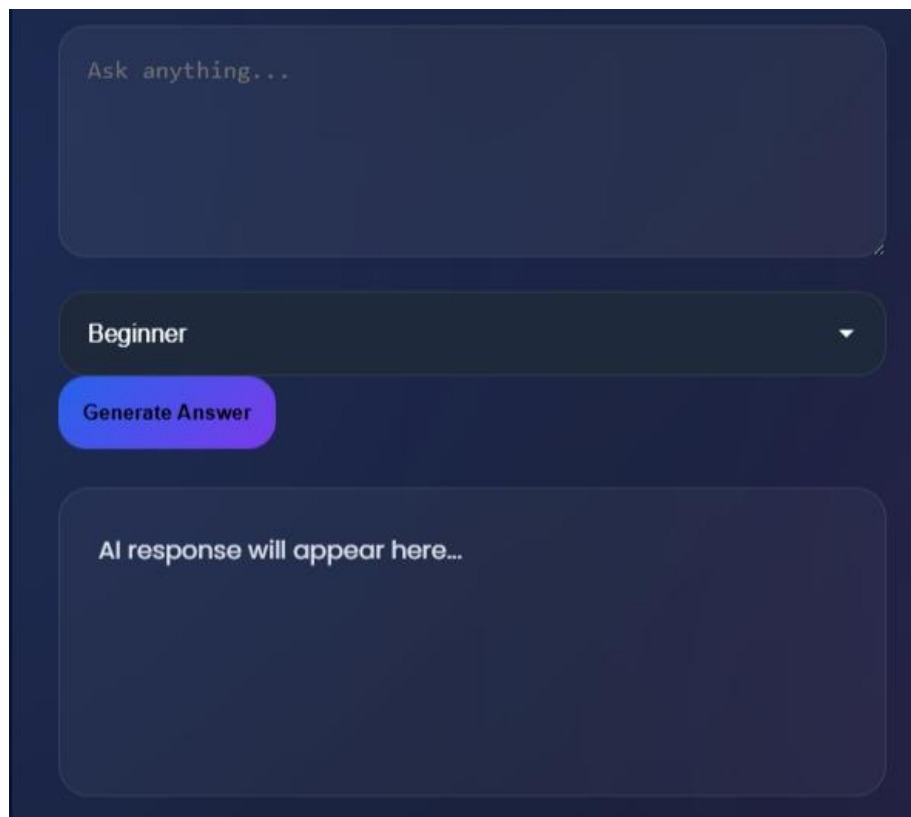
- AI Interface
  1. AI Page







## 2. AI Response View



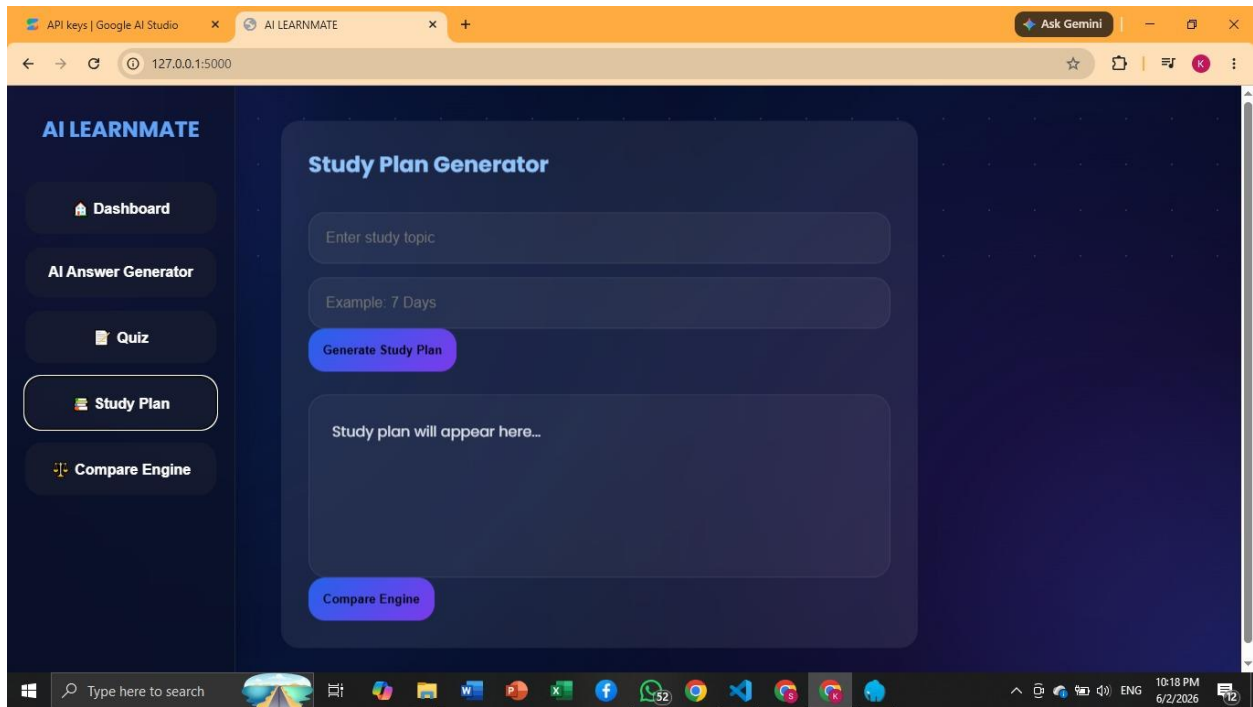
Ask anything...

Beginner

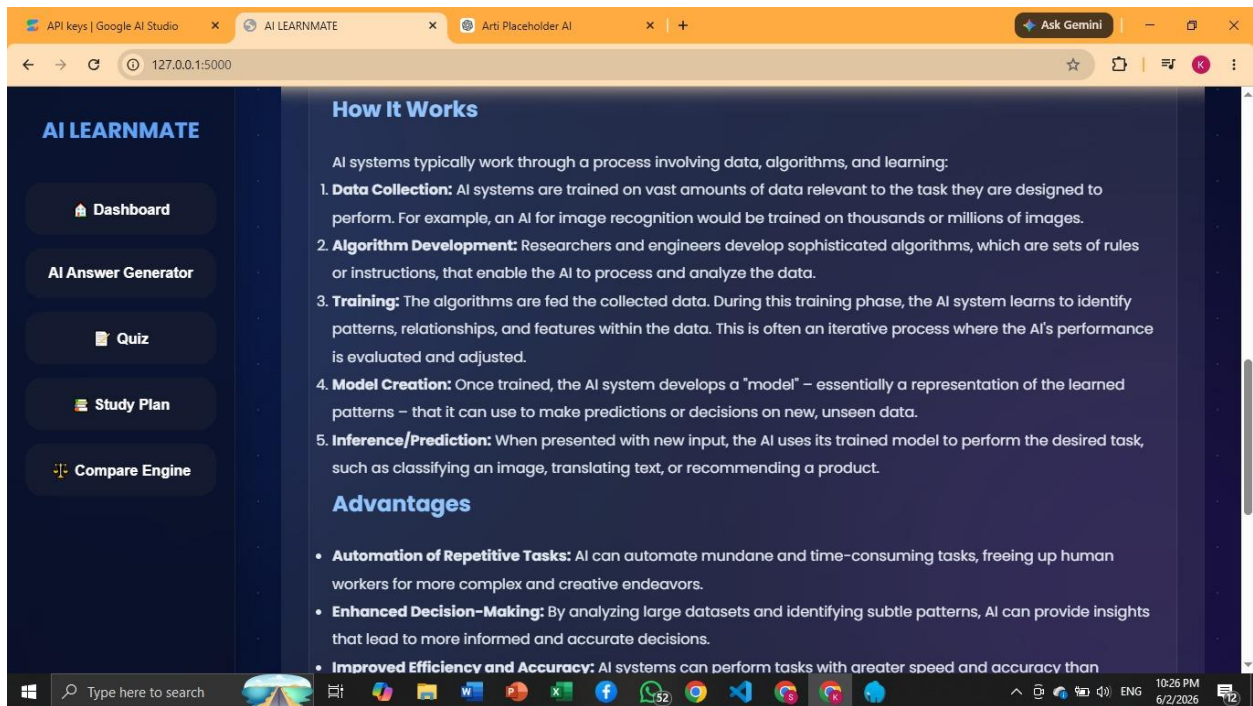


Generate Answer

AI response will appear here...



- Generated answer



## 2.2 Multi-Level Explanation Generator Design

### Objective

Provide explanations based on user knowledge level.

### Levels

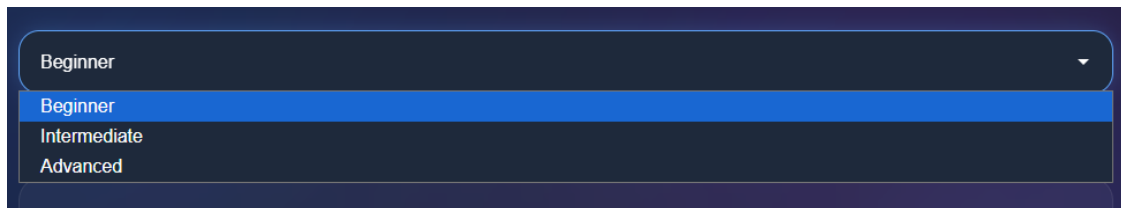
- Beginner
- Intermediate
- Advanced

### Implementation

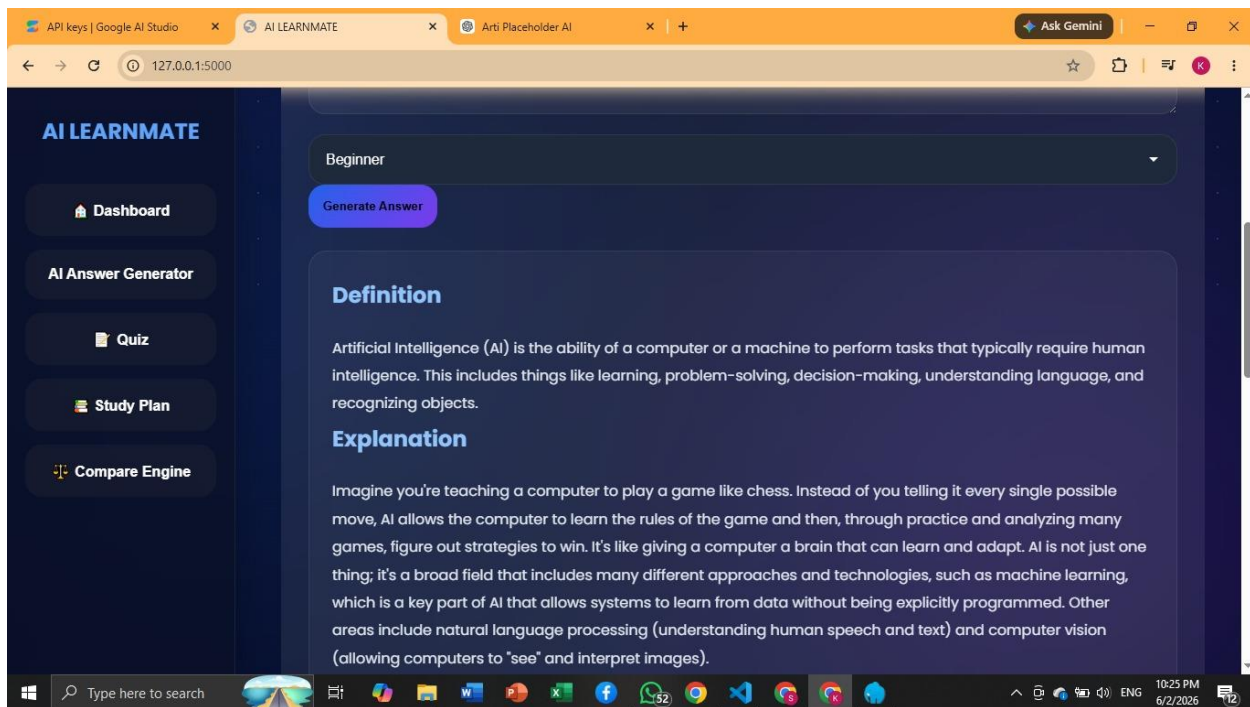
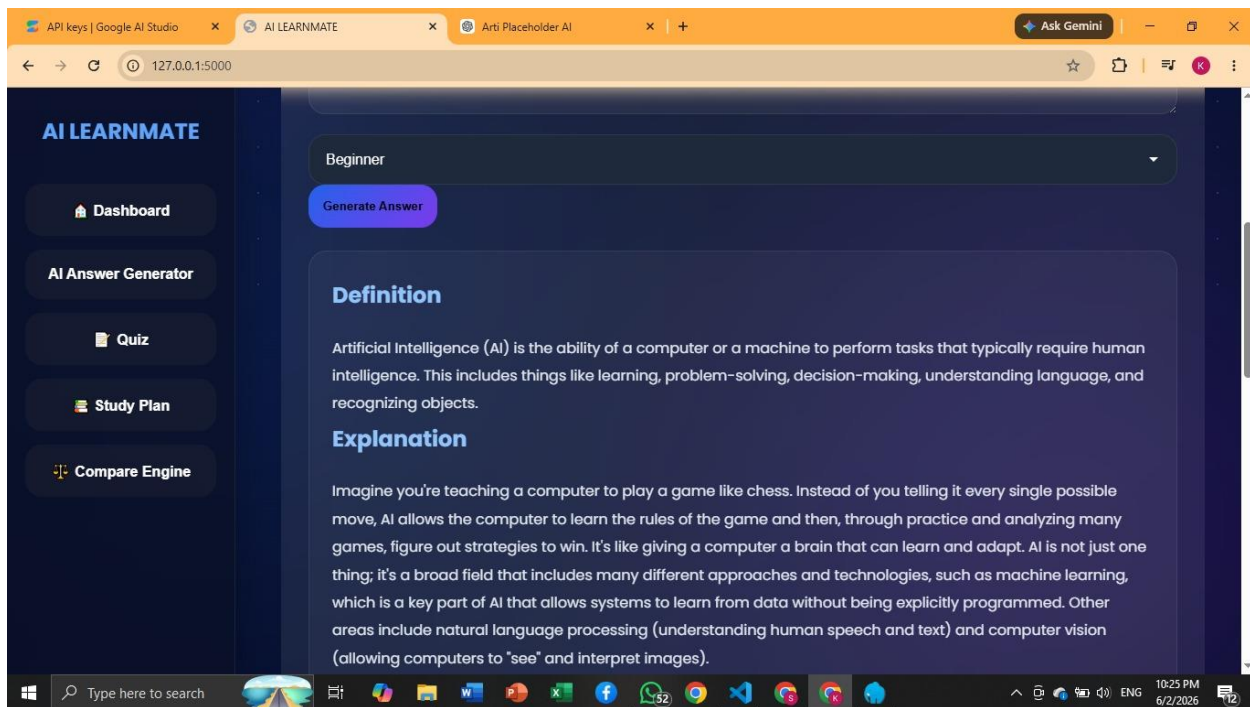
User selects level from dropdown menu.

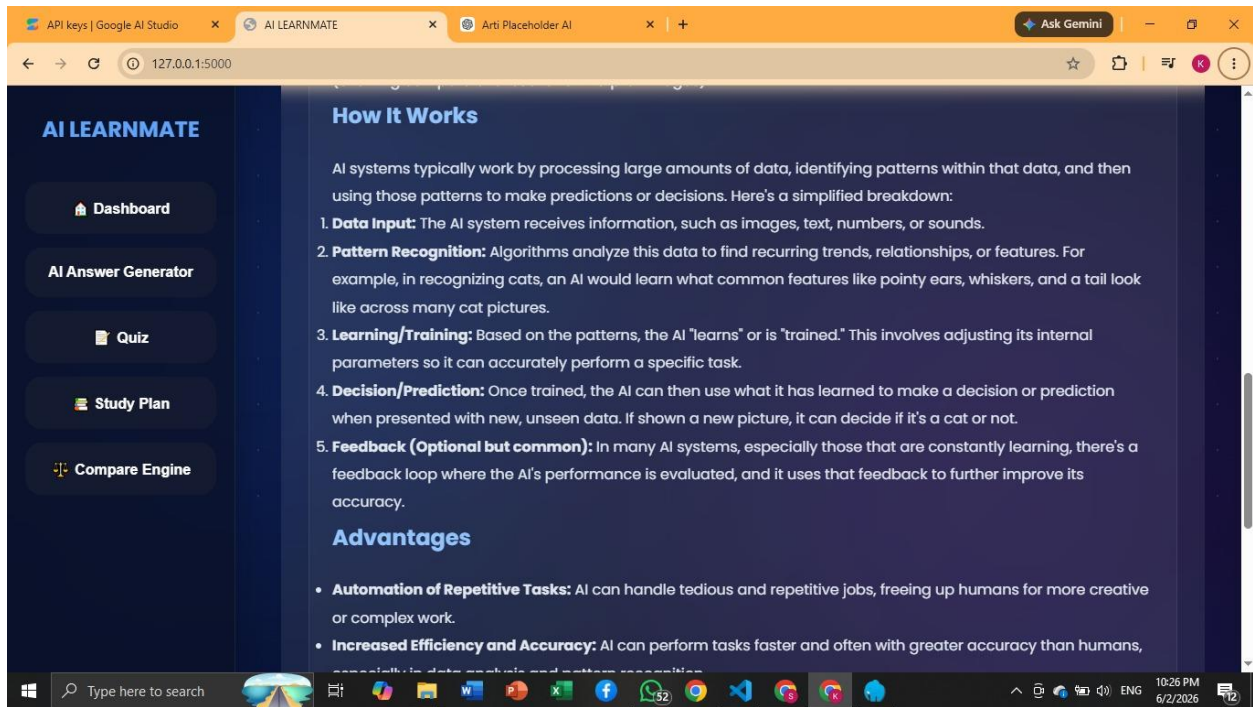
### Screenshot Required

- Level selection

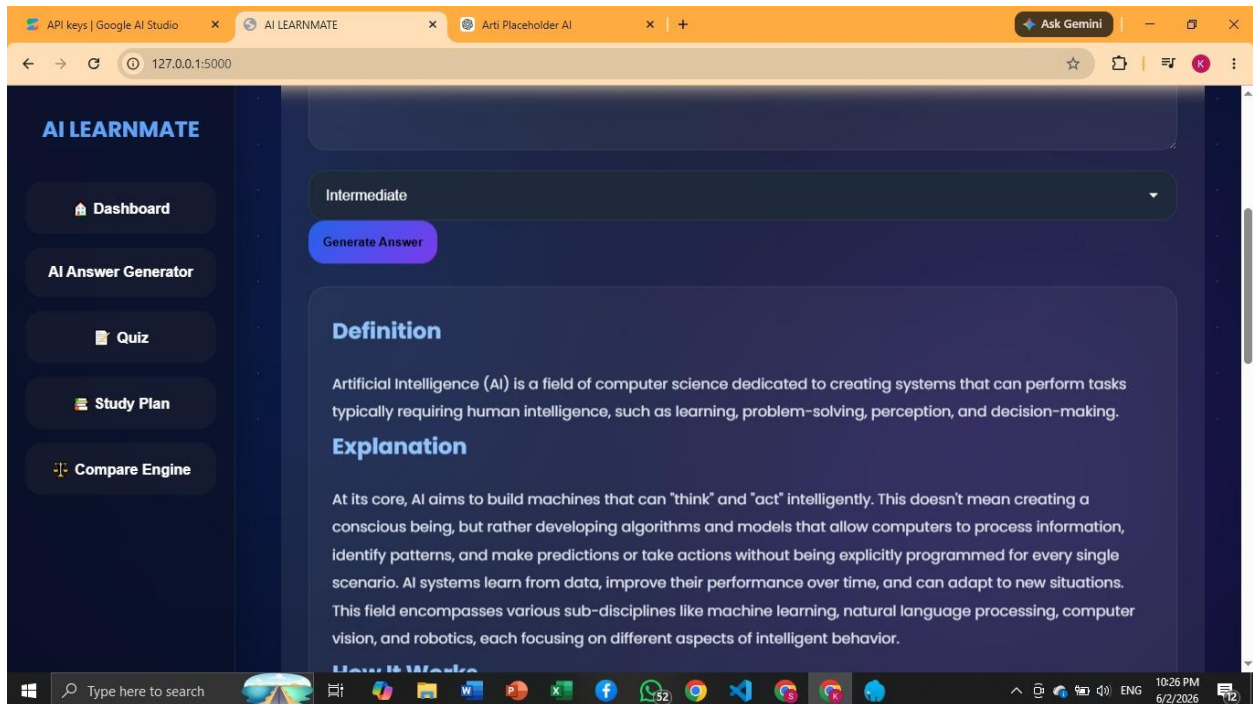


- Beginner output





- Intermediate Output





API keys | Google AI Studio

AI LEARNMATE

Arti Placeholder AI

Ask Gemini

127.0.0.1:5000

AI LEARNMATE

Dashboard

AI Answer Generator

Quiz

Study Plan

Compare Engine

How It Works

AI systems typically work through a process involving data, algorithms, and learning:

- Data Collection:** AI systems are trained on vast amounts of data relevant to the task they are designed to perform. For example, an AI for image recognition would be trained on thousands or millions of images.
- Algorithm Development:** Researchers and engineers develop sophisticated algorithms, which are sets of rules or instructions, that enable the AI to process and analyze the data.
- Training:** The algorithms are fed the collected data. During this training phase, the AI system learns to identify patterns, relationships, and features within the data. This is often an iterative process where the AI's performance is evaluated and adjusted.
- Model Creation:** Once trained, the AI system develops a "model" – essentially a representation of the learned patterns – that it can use to make predictions or decisions on new, unseen data.
- Inference/Prediction:** When presented with new input, the AI uses its trained model to perform the desired task, such as classifying an image, translating text, or recommending a product.

Advantages

- Automation of Repetitive Tasks:** AI can automate mundane and time-consuming tasks, freeing up human workers for more complex and creative endeavors.
- Enhanced Decision-Making:** By analyzing large datasets and identifying subtle patterns, AI can provide insights that lead to more informed and accurate decisions.
- Improved Efficiency and Accuracy:** AI systems can perform tasks with greater speed and accuracy than

API keys | Google AI Studio

AI LEARNMATE

Arti Placeholder AI

Ask Gemini

127.0.0.1:5000

AI LEARNMATE

Dashboard

AI Answer Generator

Quiz

Study Plan

Compare Engine

How It Works

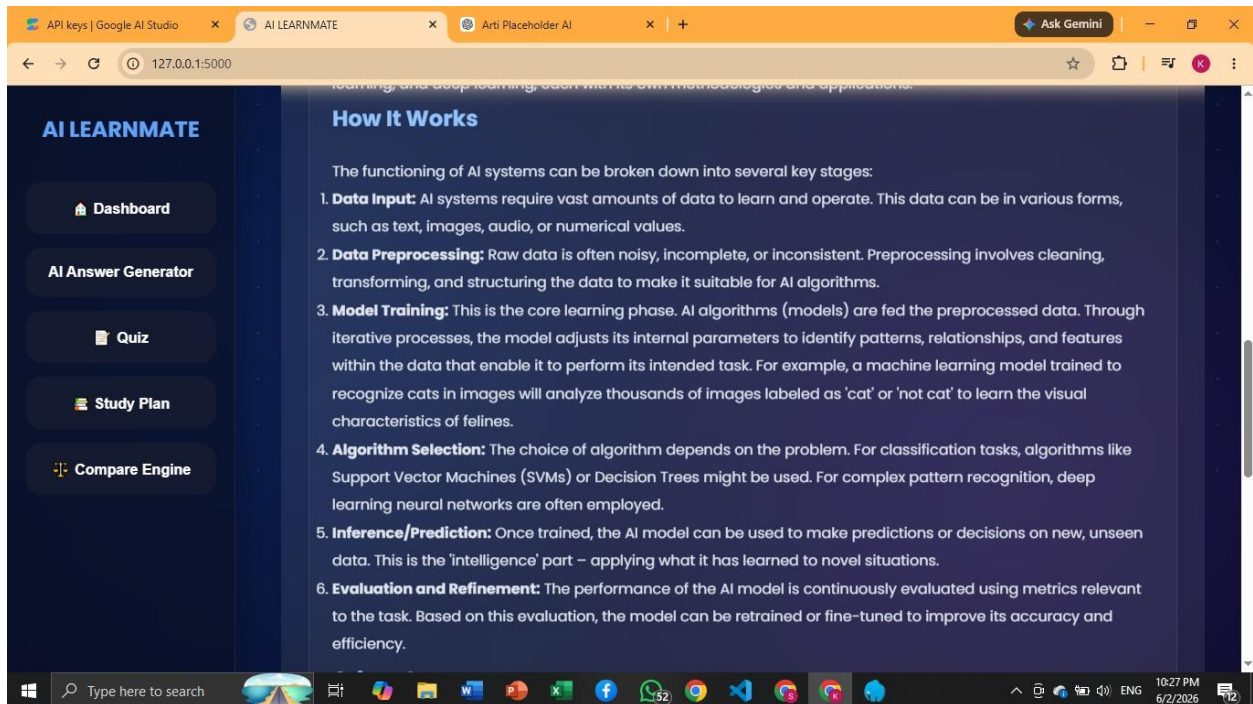
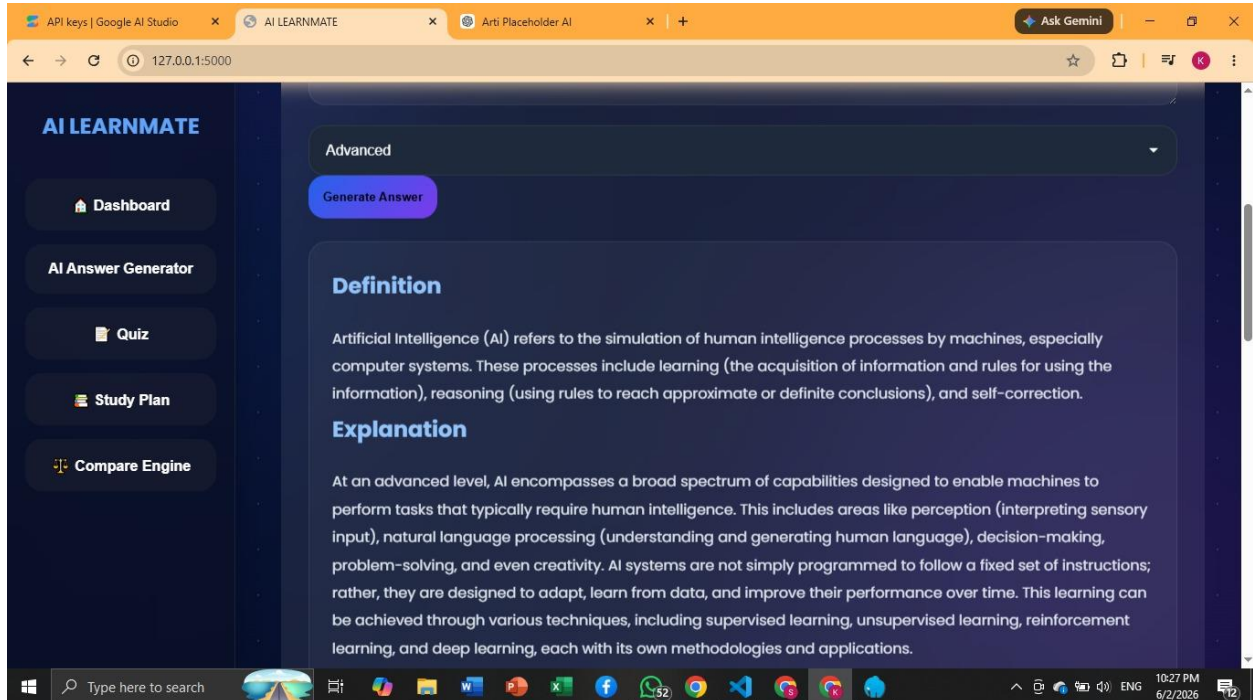
AI systems typically work through a process involving data, algorithms, and learning:

- Data Collection:** AI systems are trained on vast amounts of data relevant to the task they are designed to perform. For example, an AI for image recognition would be trained on thousands or millions of images.
- Algorithm Development:** Researchers and engineers develop sophisticated algorithms, which are sets of rules or instructions, that enable the AI to process and analyze the data.
- Training:** The algorithms are fed the collected data. During this training phase, the AI system learns to identify patterns, relationships, and features within the data. This is often an iterative process where the AI's performance is evaluated and adjusted.
- Model Creation:** Once trained, the AI system develops a "model" – essentially a representation of the learned patterns – that it can use to make predictions or decisions on new, unseen data.
- Inference/Prediction:** When presented with new input, the AI uses its trained model to perform the desired task, such as classifying an image, translating text, or recommending a product.

Advantages

- Automation of Repetitive Tasks:** AI can automate mundane and time-consuming tasks, freeing up human workers for more complex and creative endeavors.
- Enhanced Decision-Making:** By analyzing large datasets and identifying subtle patterns, AI can provide insights that lead to more informed and accurate decisions.
- Improved Efficiency and Accuracy:** AI systems can perform tasks with greater speed and accuracy than

- Advanced output





API keys | Google AI Studio x AI LEARNMATE x Arti Placeholder AI x + Ask Gemini

127.0.0.1:5000

## AI LEARNMATE

- Dashboard
- AI Answer Generator
- Quiz
- Study Plan
- Compare Engine

### How It Works

The functioning of AI systems can be broken down into several key stages:

- 1. Data Input:** AI systems require vast amounts of data to learn and operate. This data can be in various forms, such as text, images, audio, or numerical values.
- 2. Data Preprocessing:** Raw data is often noisy, incomplete, or inconsistent. Preprocessing involves cleaning, transforming, and structuring the data to make it suitable for AI algorithms.
- 3. Model Training:** This is the core learning phase. AI algorithms (models) are fed the preprocessed data. Through iterative processes, the model adjusts its internal parameters to identify patterns, relationships, and features within the data that enable it to perform its intended task. For example, a machine learning model trained to recognize cats in images will analyze thousands of images labeled as 'cat' or 'not cat' to learn the visual characteristics of felines.
- 4. Algorithm Selection:** The choice of algorithm depends on the problem. For classification tasks, algorithms like Support Vector Machines (SVMs) or Decision Trees might be used. For complex pattern recognition, deep learning neural networks are often employed.
- 5. Inference/Prediction:** Once trained, the AI model can be used to make predictions or decisions on new, unseen data. This is the 'intelligence' part – applying what it has learned to novel situations.
- 6. Evaluation and Refinement:** The performance of the AI model is continuously evaluated using metrics relevant to the task. Based on this evaluation, the model can be retrained or fine-tuned to improve its accuracy and efficiency.

API keys | Google AI Studio x AI LEARNMATE x Arti Placeholder AI x + Ask Gemini

127.0.0.1:5000

## AI LEARNMATE

- Dashboard
- AI Answer Generator
- Quiz
- Study Plan
- Compare Engine

### Advantages

- **Automation of Complex Tasks:** AI can automate repetitive, time-consuming, or dangerous tasks, freeing up human resources for more strategic or creative endeavors.
- **Enhanced Decision-Making:** By analyzing large datasets and identifying subtle patterns, AI can provide insights that lead to more informed and accurate decision-making.
- **Improved Efficiency and Productivity:** AI-powered systems can operate 24/7 without fatigue, leading to significant gains in operational efficiency and overall productivity.
- **Personalization and Customization:** AI can tailor experiences, products, and services to individual user preferences, leading to greater satisfaction.

### Example

Consider the recommendation engines used by streaming services like Netflix or e-commerce platforms like Amazon. These AI systems analyze your viewing or purchasing history, the behavior of similar users, and various other data points. Based on this analysis, they predict what movies you might want to watch next or what products you might be interested in buying. This is an example of predictive AI and recommendation systems, a widely used application that enhances user experience and drives engagement.

### Summary

- AI simulates human cognitive functions like learning, reasoning, and problem-solving in machines.

## 2.3 Testing and Refinement

### Objective

Improve answer quality.

### Activities

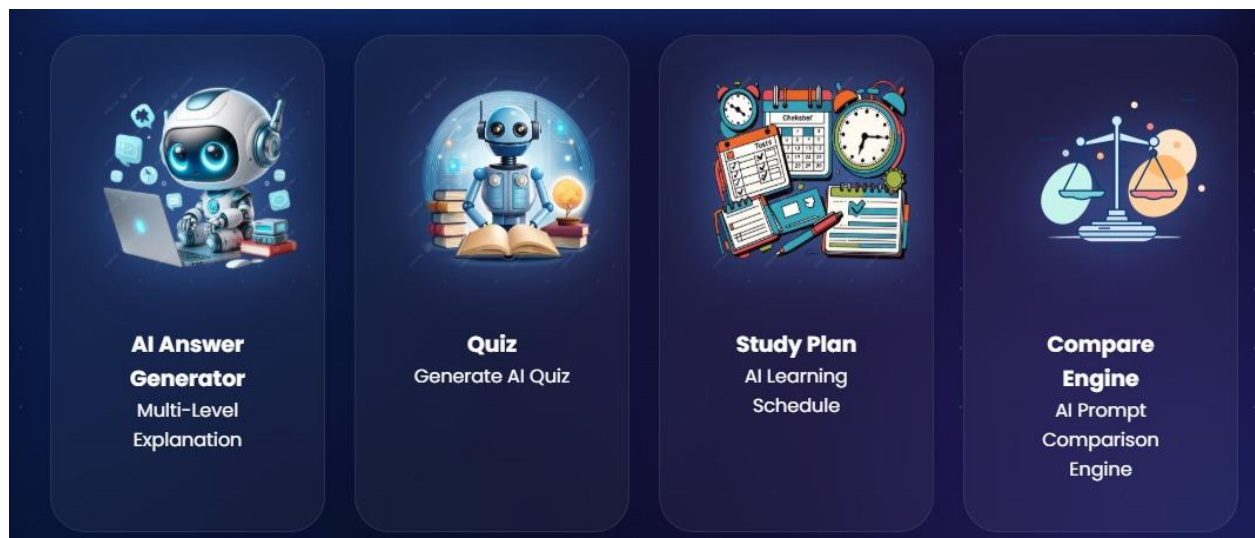
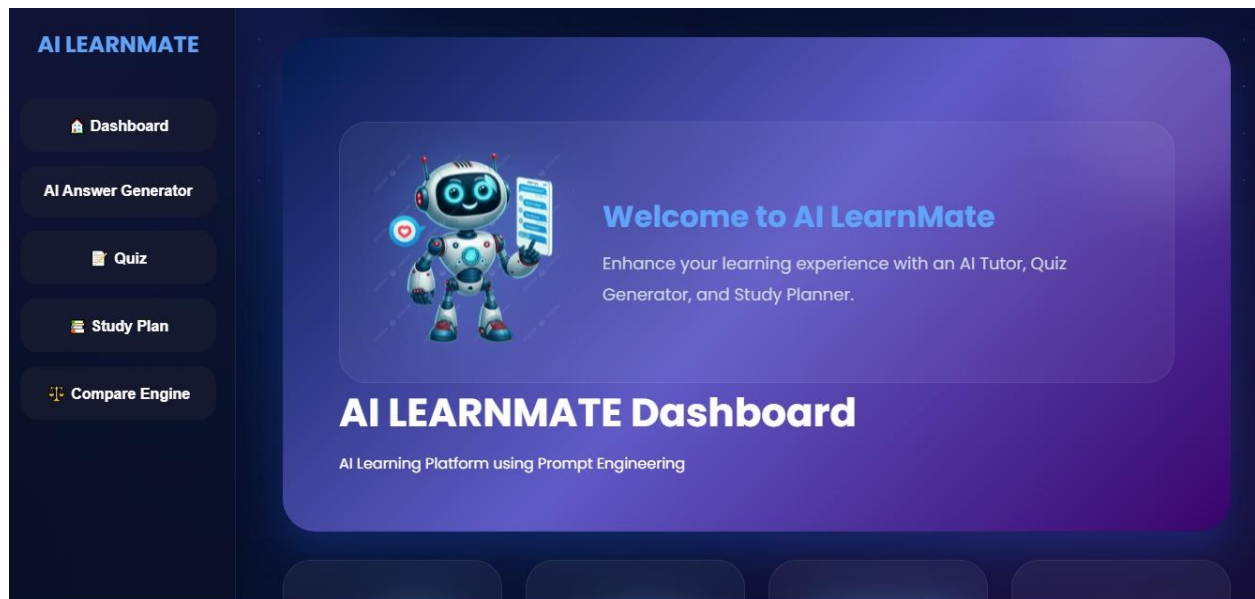
1. Test multiple topics.
2. Improve prompts.
3. Remove Markdown outputs.
4. Force HTML structure.

### Screenshot Required

- Before refinement



- After refinement



## M3. Study Plan Generator and Quiz Generator

### 3.1 Study Plan Generator Development

#### Objective

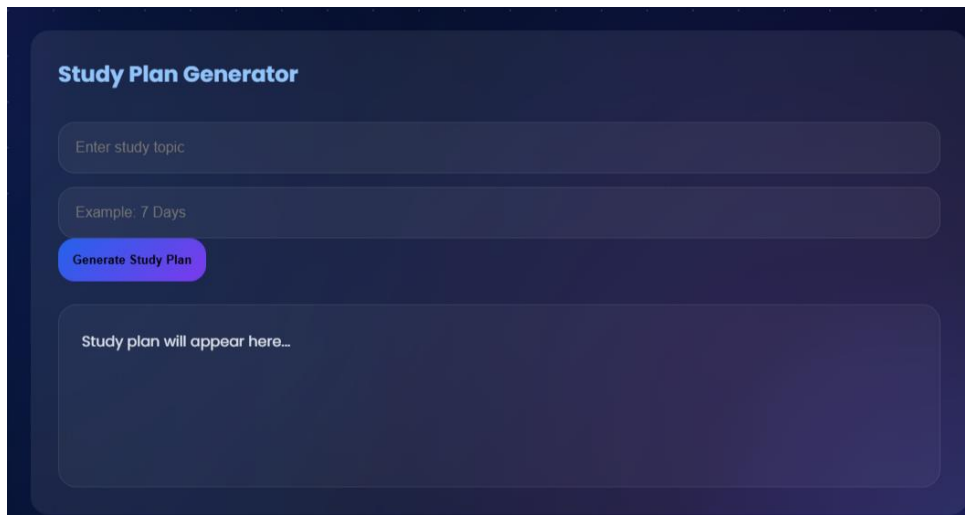
Generate learning schedules automatically.

#### Features

- Topic input
- Duration input
- Daily learning objectives
- Practice tasks

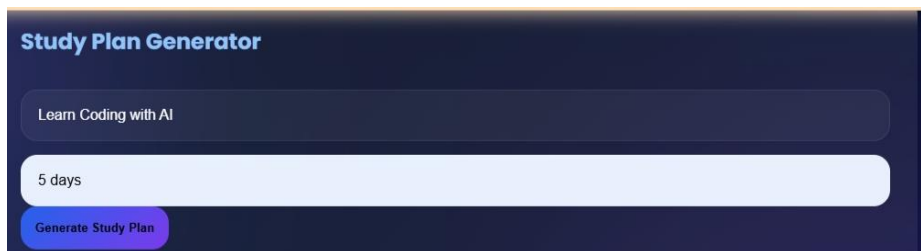
## Screenshot Required

- Study Plan page

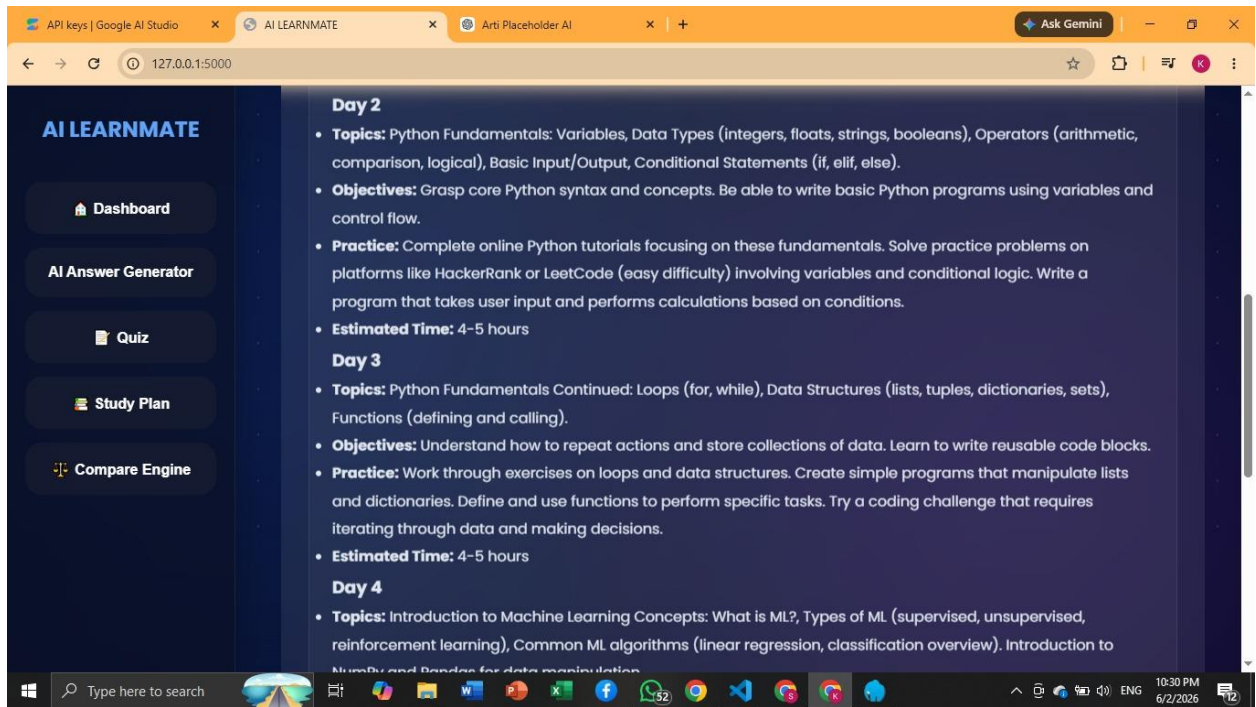


The screenshot shows a dark-themed web interface for a 'Study Plan Generator'. At the top, the title 'Study Plan Generator' is displayed in a light blue font. Below the title, there are two input fields: the first is labeled 'Enter study topic' and the second is labeled 'Example: 7 Days'. A blue button with the text 'Generate Study Plan' is positioned below the second input field. At the bottom of the form, there is a large, empty rectangular box with the placeholder text 'Study plan will appear here...' centered inside it.

- Generated study plan



This screenshot shows the same 'Study Plan Generator' interface, but with the input fields filled. The first input field now contains the text 'Learn Coding with AI', and the second input field contains '5 days'. The blue 'Generate Study Plan' button remains visible below the second field. The output area at the bottom is not visible in this screenshot.



## 3.2 Quiz Generator Development

### Objective

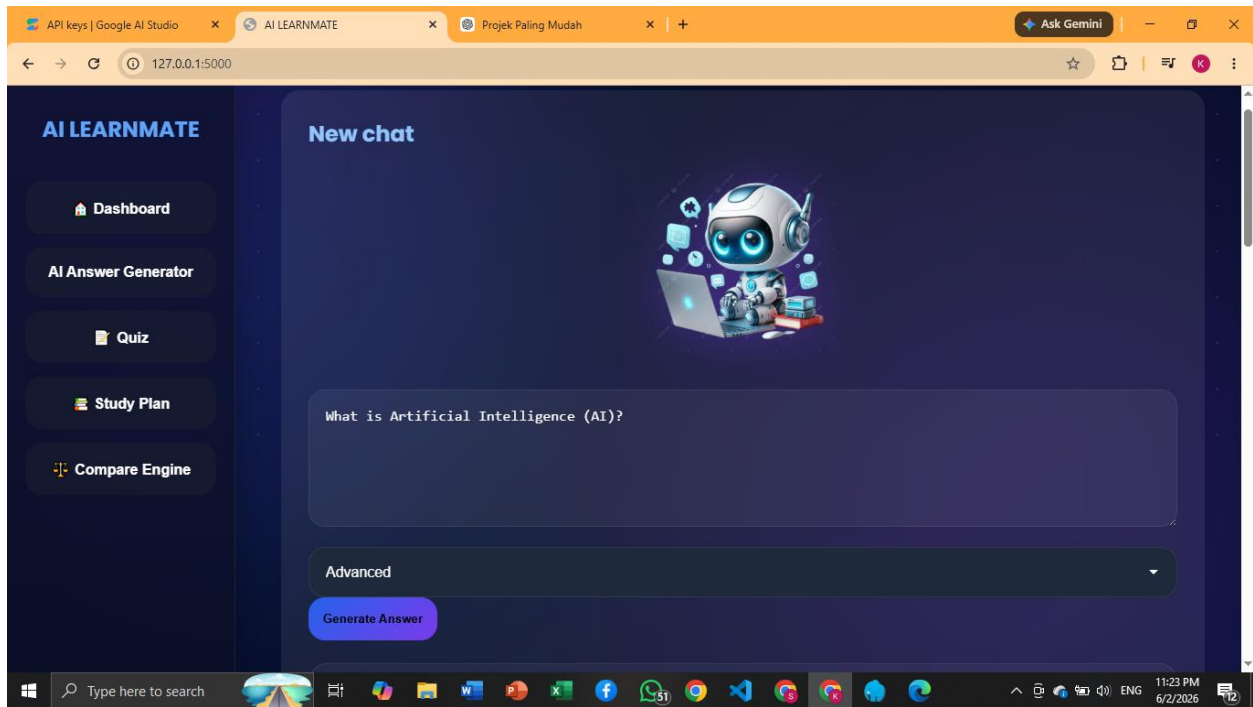
Generate multiple-choice quizzes automatically.

### Features

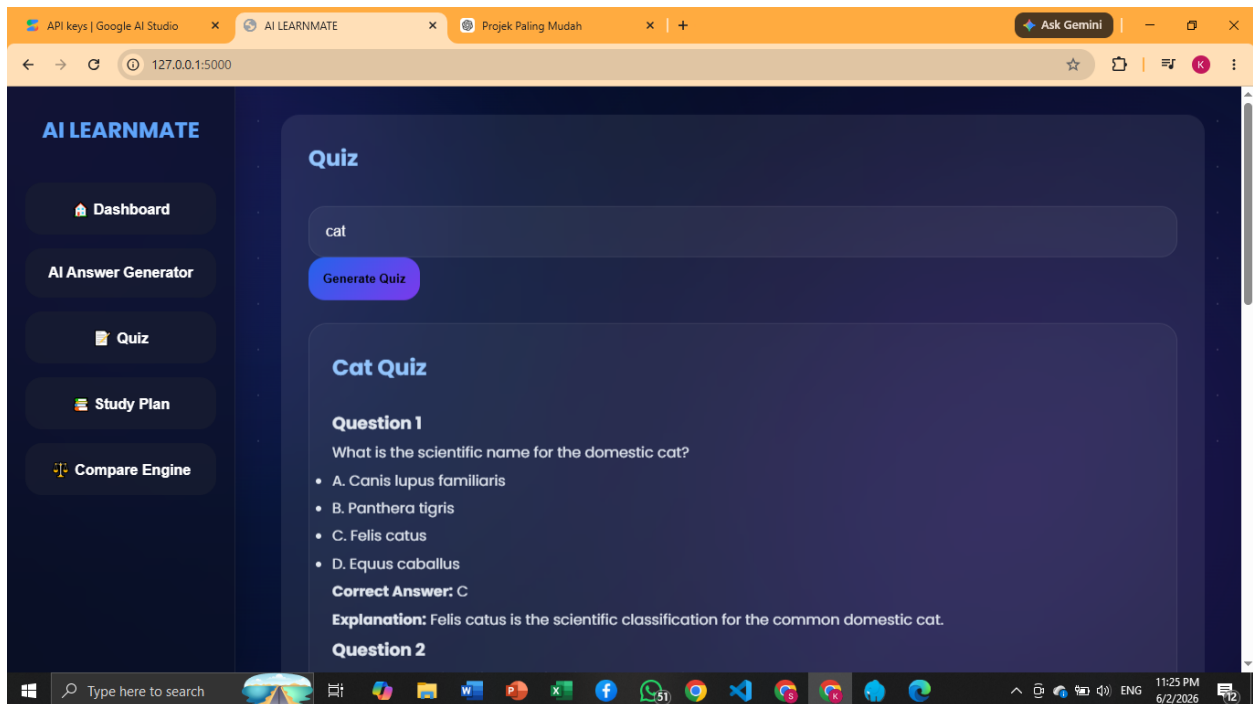
- Topic selection
- Number of questions
- Multiple-choice answers
- Correct answer feedback

### Screenshot Required

- Quiz form



- Generated questions



### 3.3 Few-Shot Prompting Integration

#### Objective

Improve consistency using examples.

#### Example Structure

Topic: Internet

Definition:

Internet is a global network...

Example:

Google, YouTube, Email

#### Screenshot Required

- Few-shot prompt

```
• # =====  
• # FEW SHOT PROMPT  
• # =====  
• fewshot_prompt = f"""  
• Example:  
•  
• Topic: Internet  
•  
• <h2>Definition</h2>  
• <p>  
• The Internet is a global network that connects computers and devices worldwide.  
• </p>  
•  
• <h2>How It Works</h2>  
• <p>  
• Devices communicate through networks using internet protocols.  
• </p>  
•  
• <h2>Advantages</h2>  
• <ul>  
• <li>Fast communication</li>  
• <li>Access to information</li>  
• <li>Online services</li>  
• </ul>  
•  
• <h2>Example</h2>
```

- `<p>`
- Google, YouTube, Email, and Online Banking.
- `</p>`
- 
- `<h2>Summary</h2>`
- `<ul>`
- `<li>Connects people globally</li>`
- `<li>Provides information</li>`
- `<li>Supports digital services</li>`
- `</ul>`
- 
- Now explain:
- 
- `{question}`
- 
- Use the SAME format.
- 
- Rules:
- - Use English only.
- - Return HTML only.
- - No Markdown.
- 
- """"

- Output generated

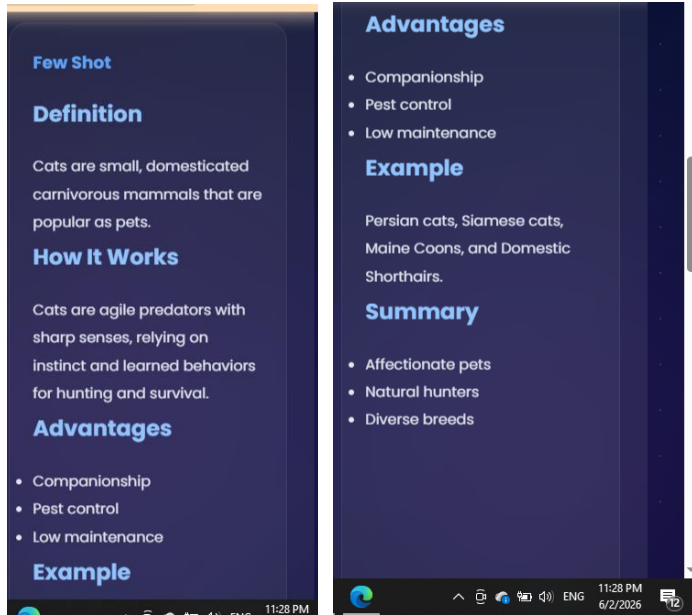
**Prompt Engineering Compare Engine**

Compare Zero-Shot, Structured Prompt, and Few-Shot Prompting results.

Explain about cats?

Compare





## M5. Prompt Comparison Engine and Optimization

### 5.1 Prompt Comparison Engine Development

#### Objective

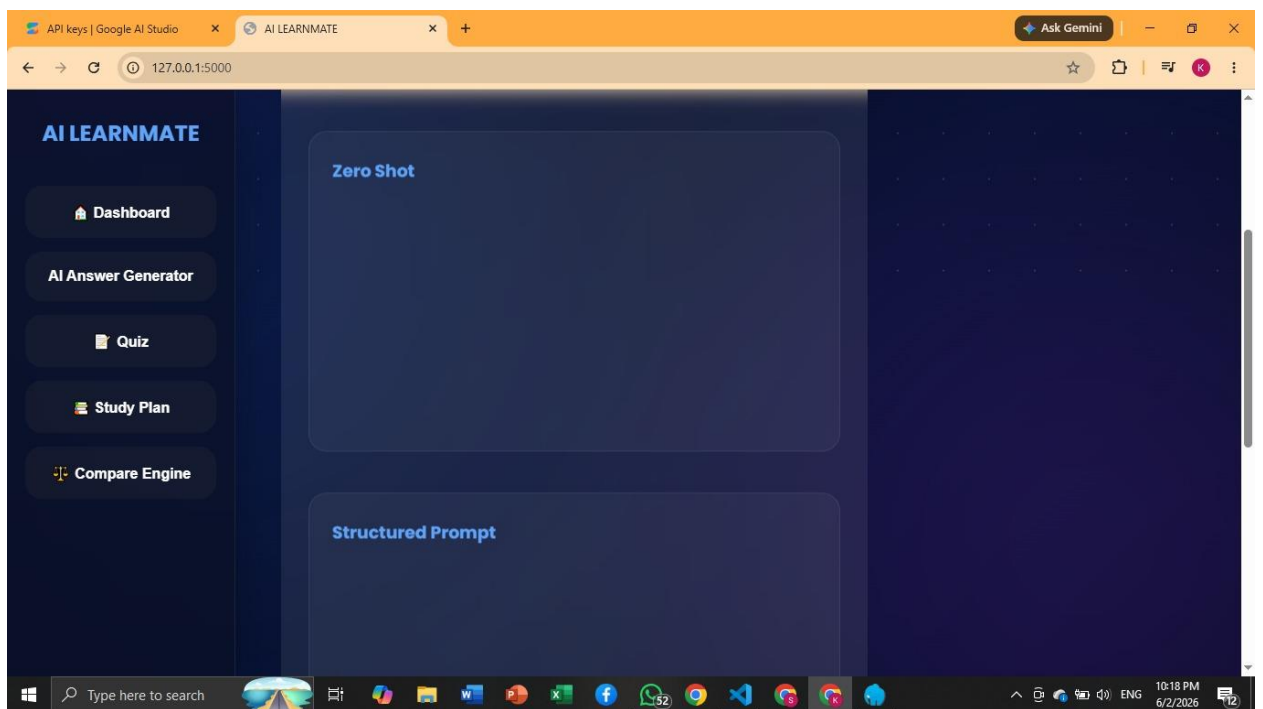
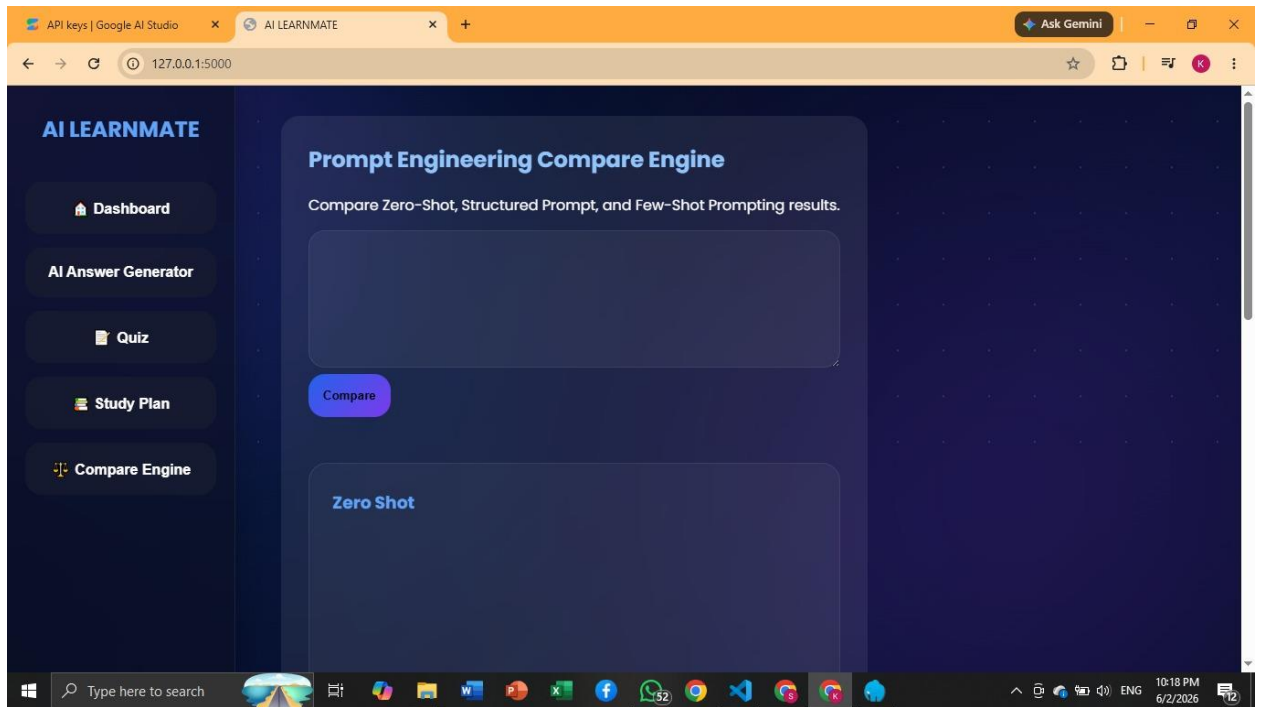
Compare multiple prompting techniques.

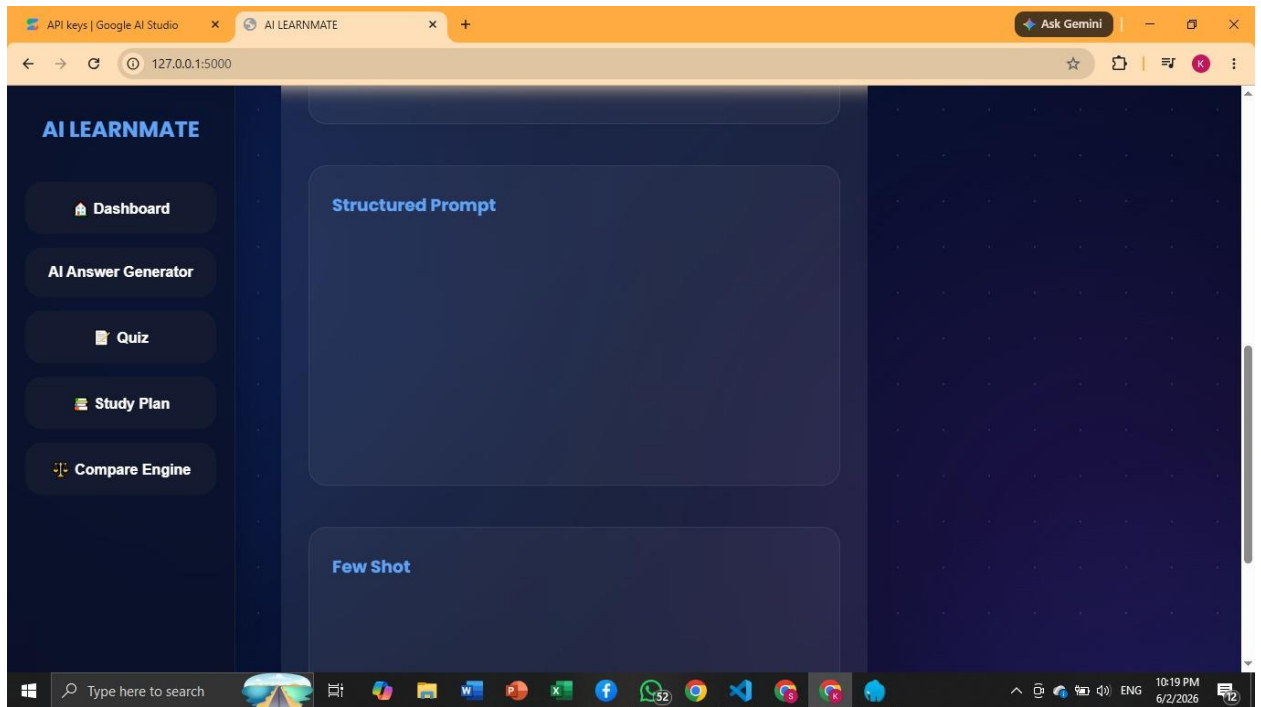
#### Techniques

1. Zero-Shot Prompting
2. Structured Prompting
3. Few-Shot Prompting

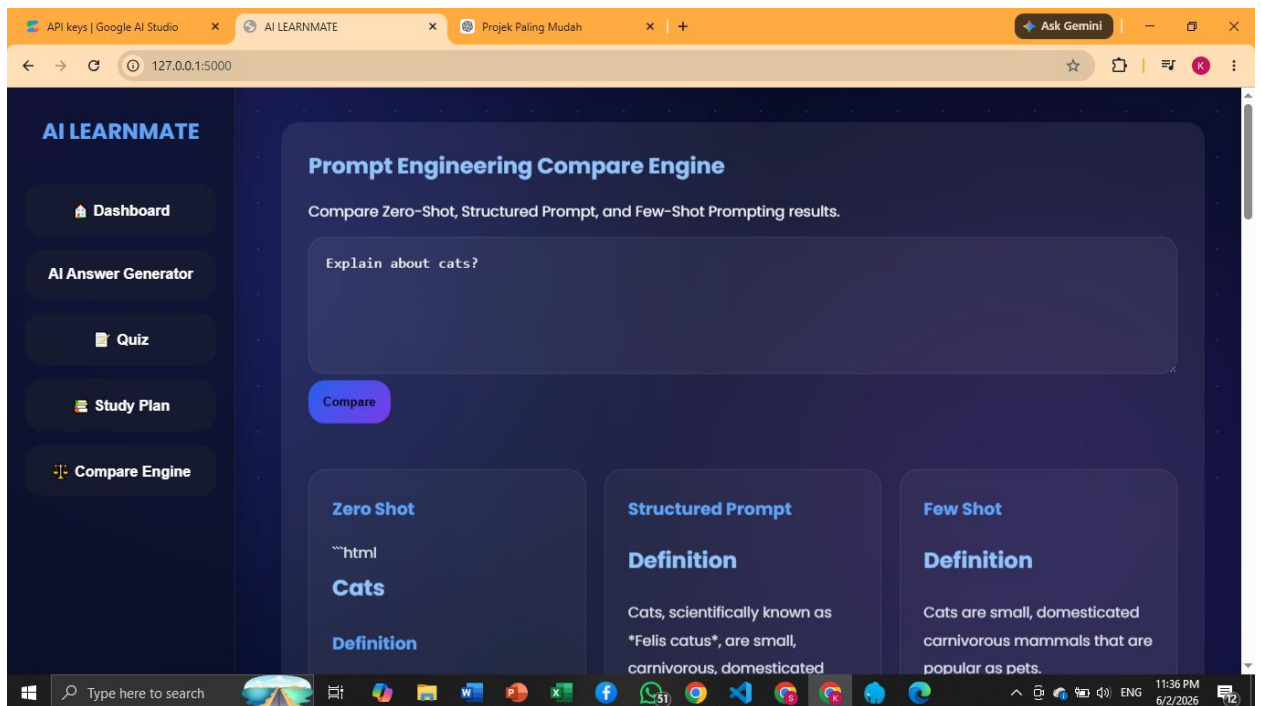
#### Screenshot Required

- Compare Engine interface

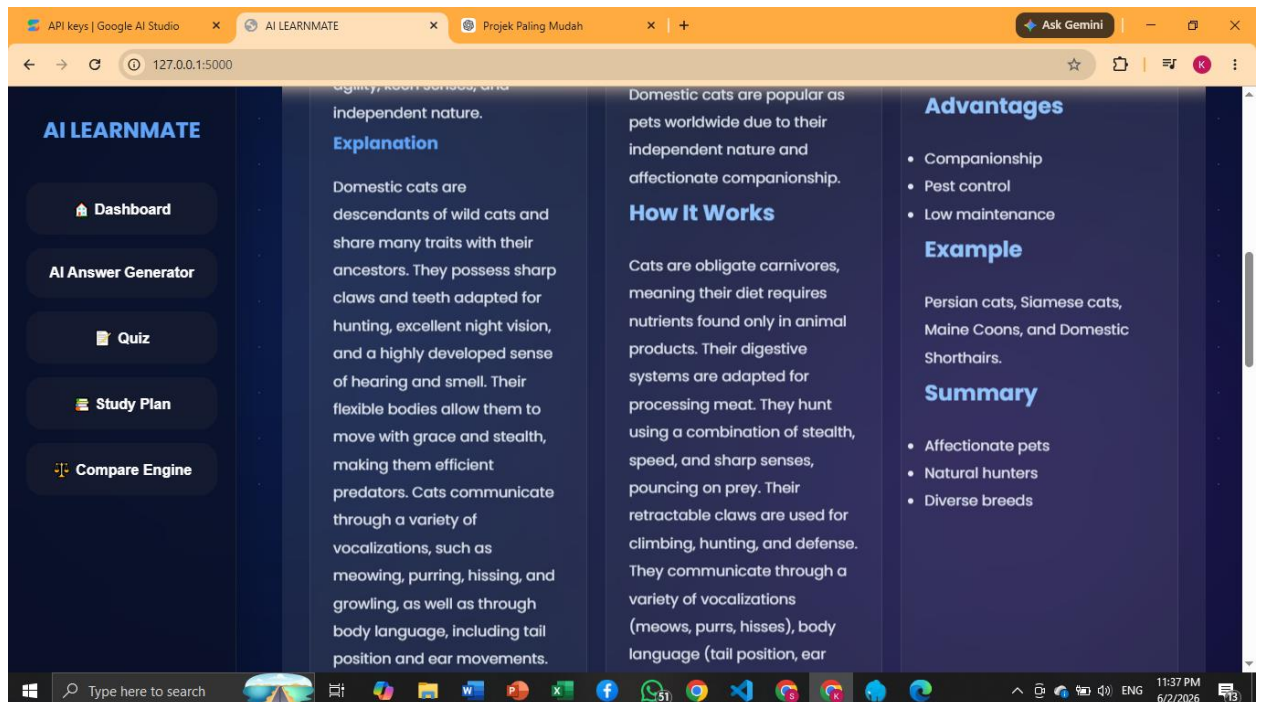
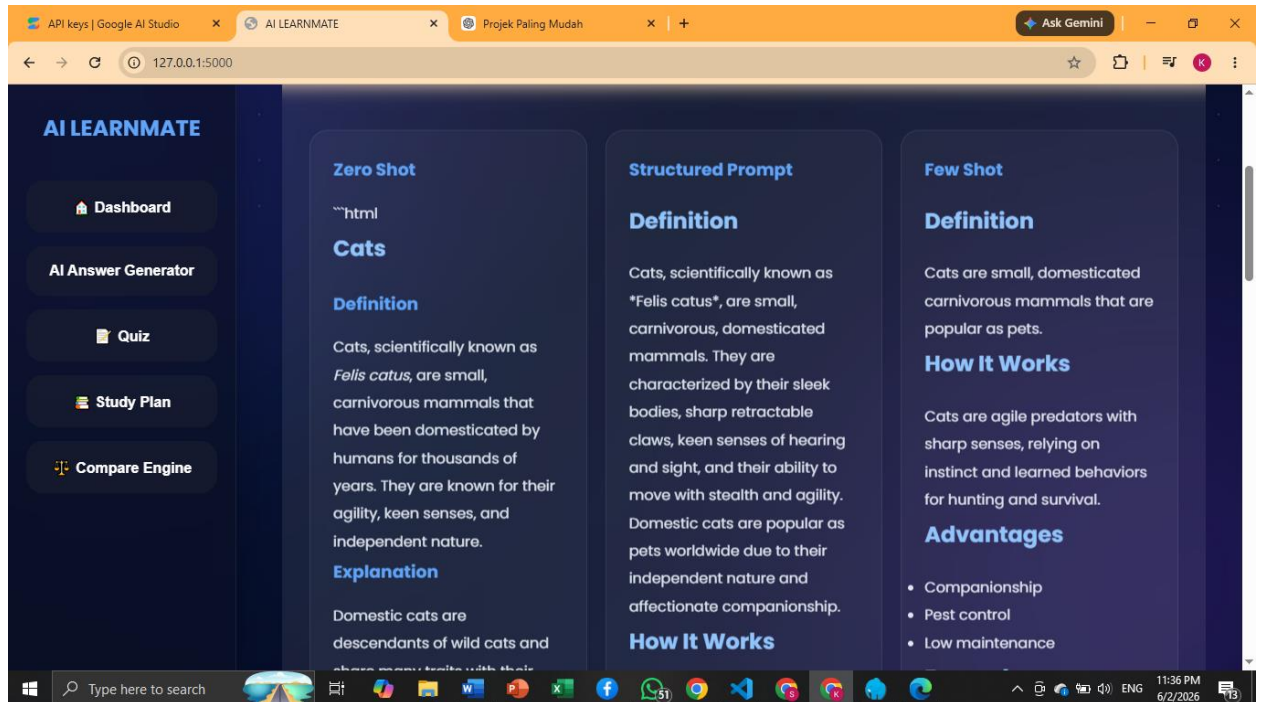


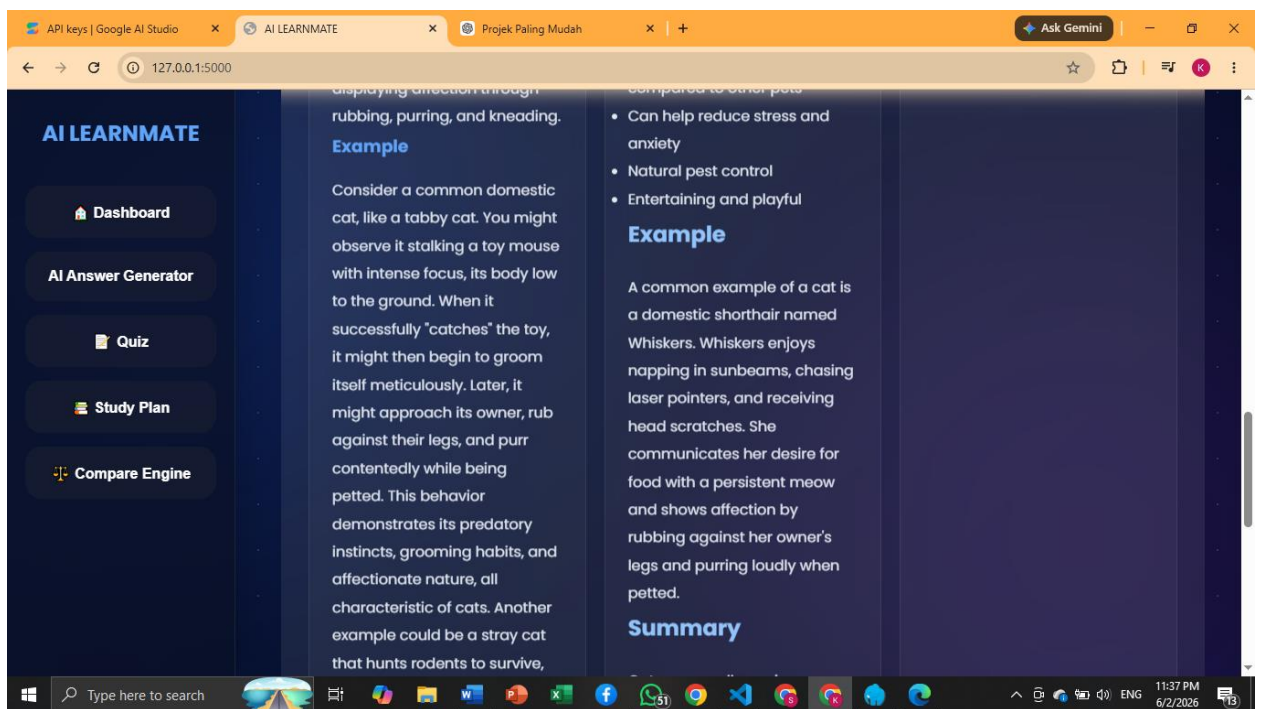
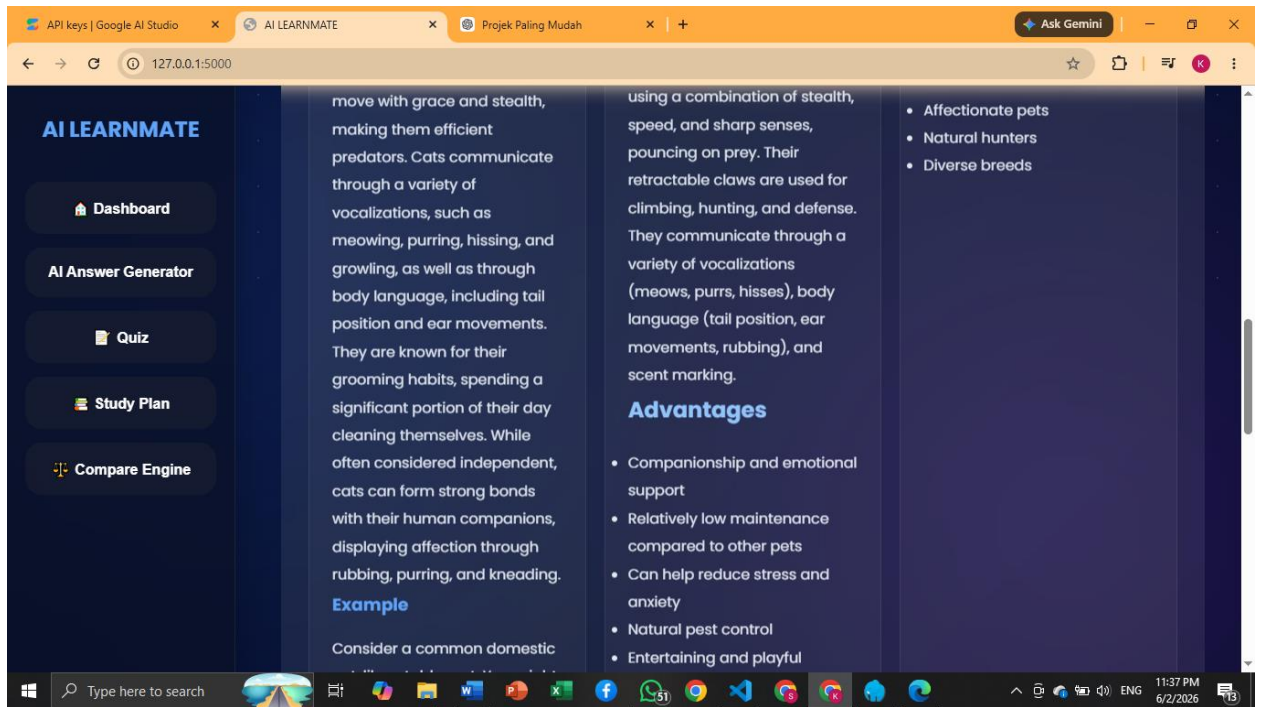


- Input topic

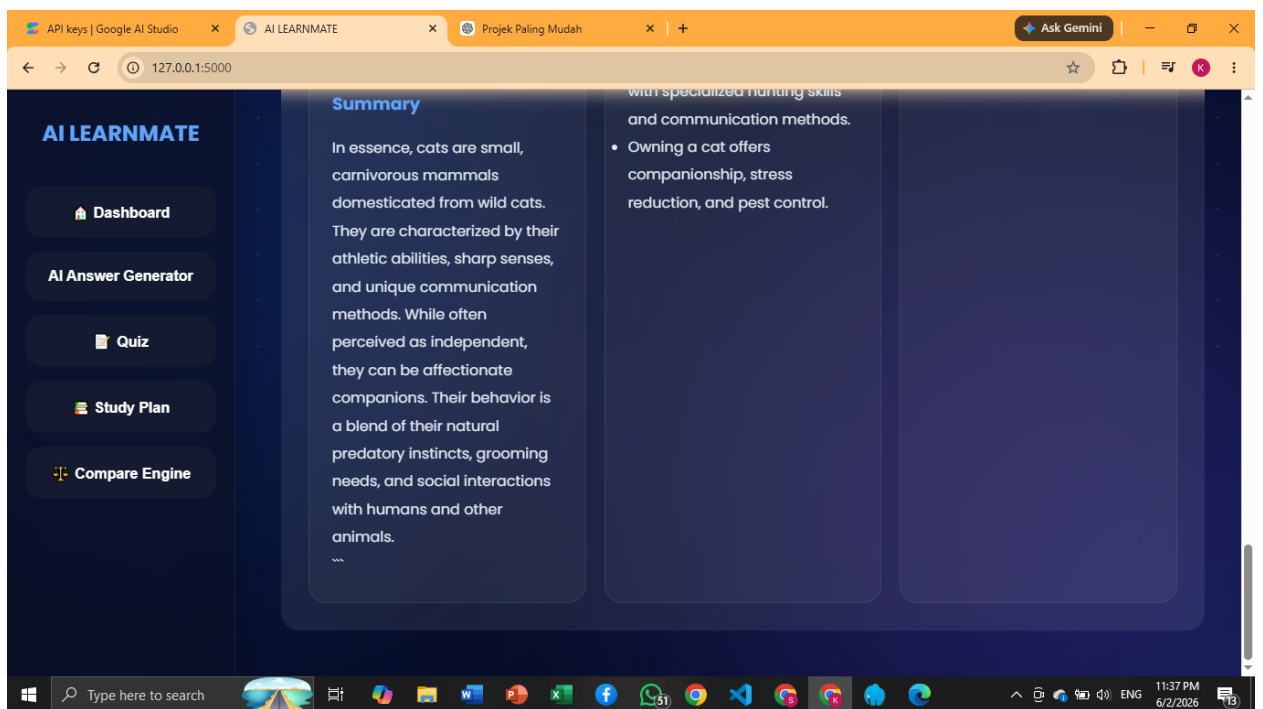
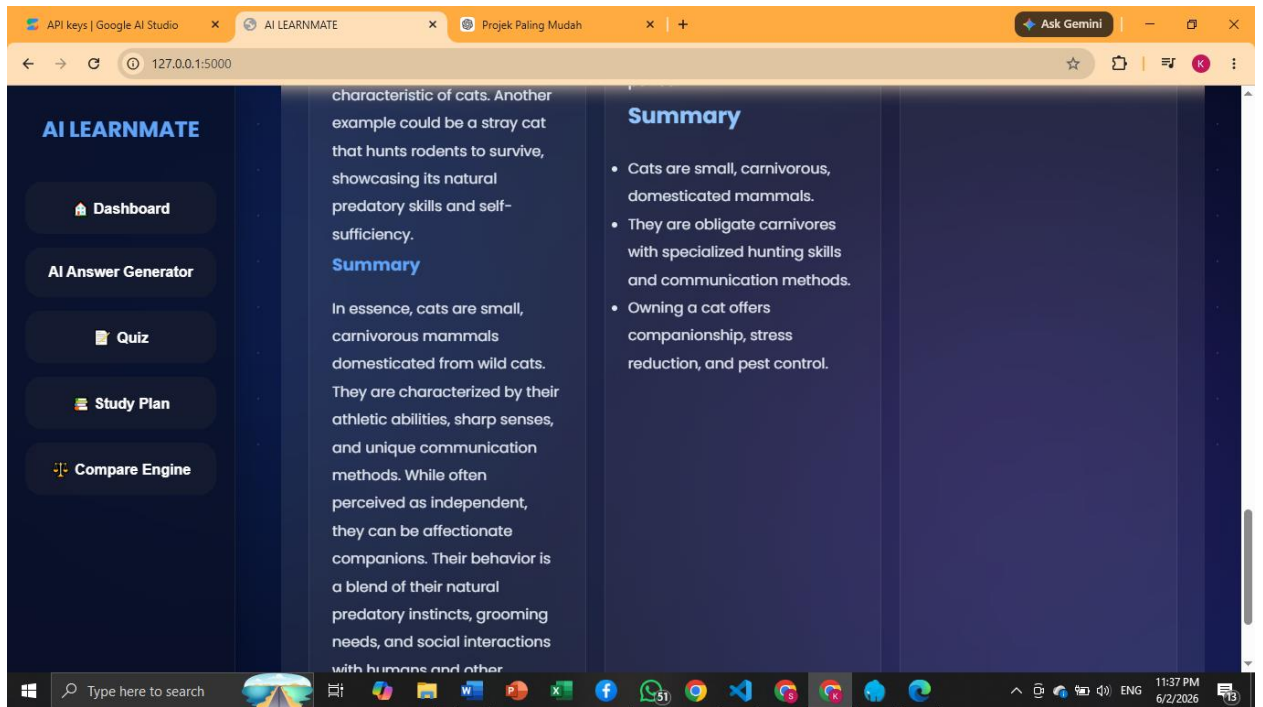


- Generated outputs









## 5.2 Prompt Optimization

### Objective

Improve prompt performance.

### Activities

1. Reduce ambiguity.
2. Improve output structure.
3. Increase consistency.

### Screenshot Required

- Original prompt
  - Optimized prompt
  - Result comparison
- 

## 5.3 Performance Evaluation

### Evaluation Criteria

- Accuracy
- Consistency
- Readability
- Response Structure

### Comparison Table

Technique Accuracy Structure Consistency

|            |        |        |           |
|------------|--------|--------|-----------|
| Zero-Shot  | Medium | Low    | Medium    |
| Structured | High   | High   | High      |
| Few-Shot   | High   | Medium | Very High |

### Screenshot Required

- Final comparison results

## **M6. Finalization, Documentation, and Presentation**

### **6.1 Final Project Report Writing**

#### **Contents**

- Introduction
- Methodology
- Implementation
- Results
- Conclusion

#### **Screenshot Required**

- Final report pages
- 

### **6.2 Prompt Template Library Creation**

#### **Templates**

1. Structured Answer Prompt
2. Study Plan Prompt
3. Quiz Prompt
4. Few-Shot Prompt
5. Prompt Comparison Prompt

#### **Screenshot Required**

- Prompt library
- 

### **6.3 Presentation Preparation**

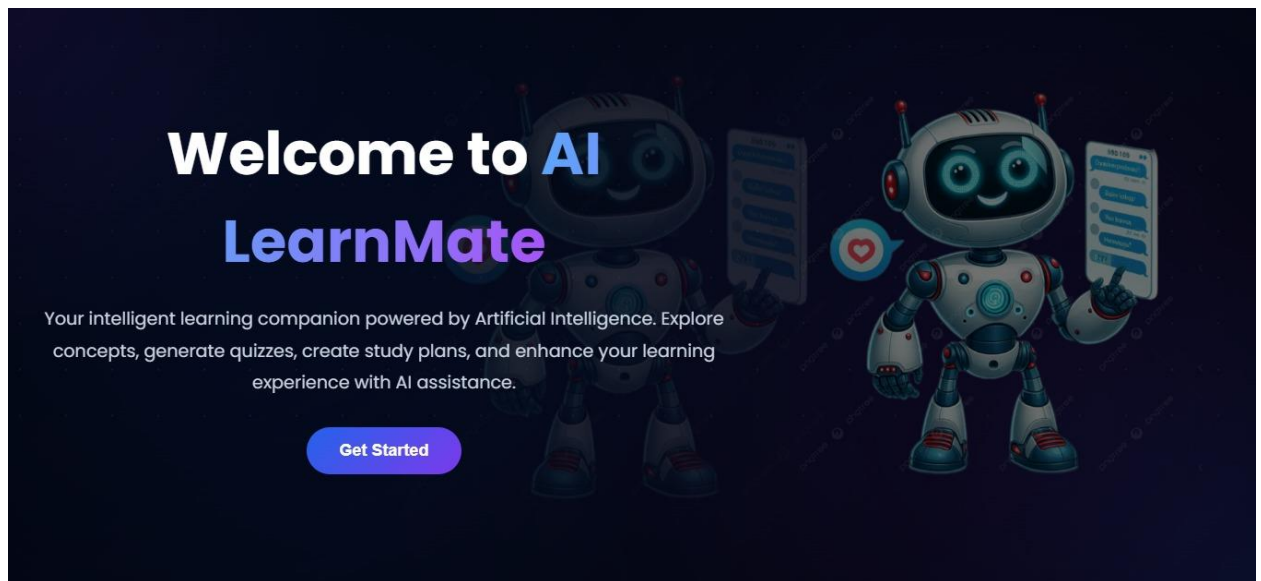
#### **Presentation Contents**

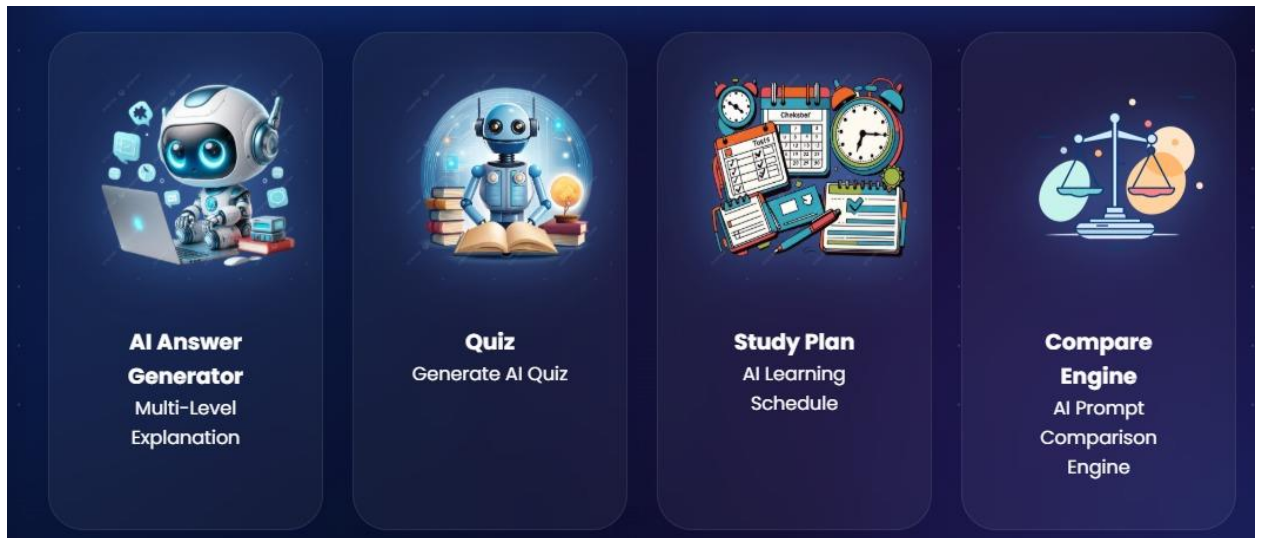
1. Project Overview
2. System Architecture
3. Features
4. Prompt Engineering Techniques
5. Demonstration
6. Results
7. Conclusion



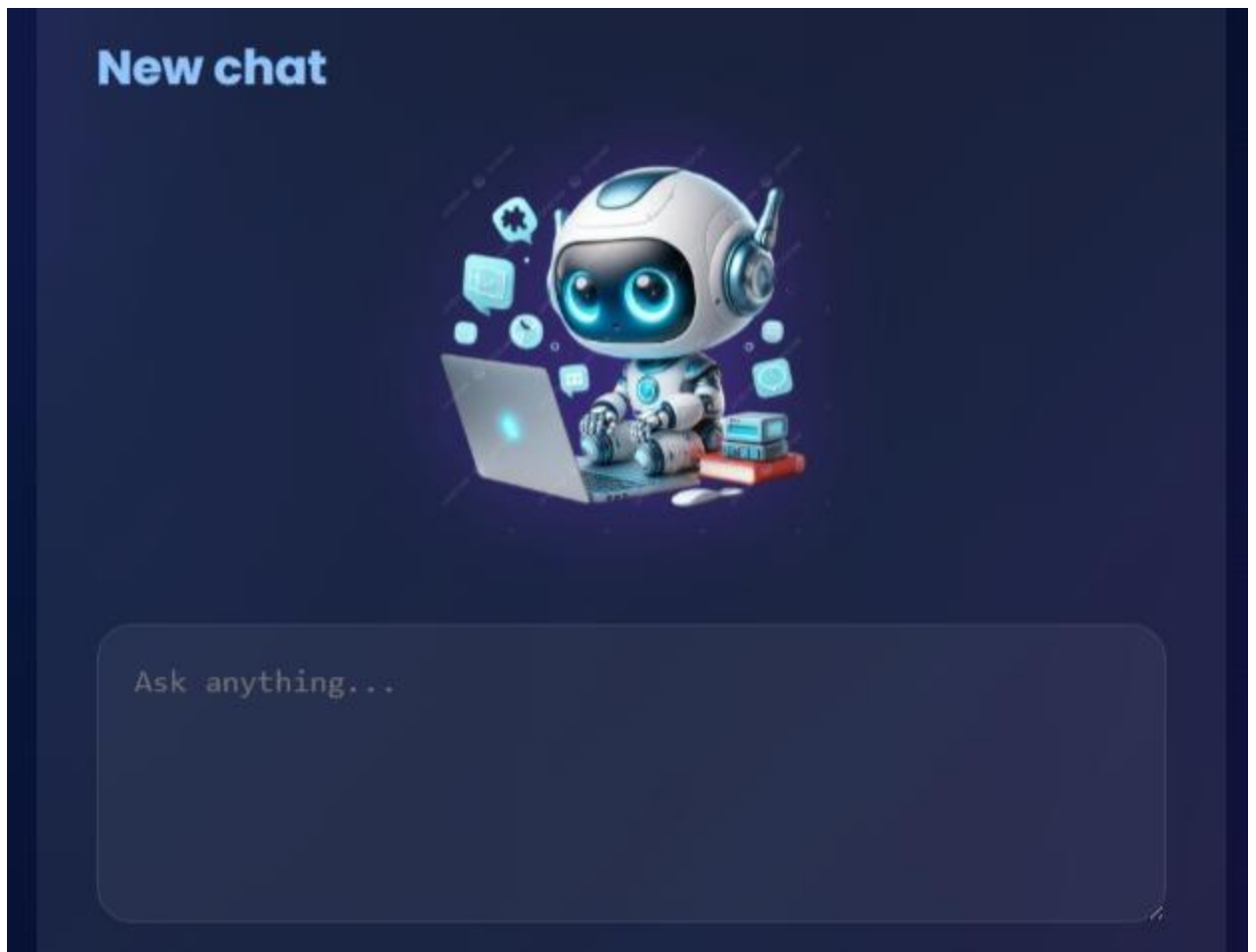
## Screenshot Required

- Final dashboard





- AI Answer Generator



Ask anything...

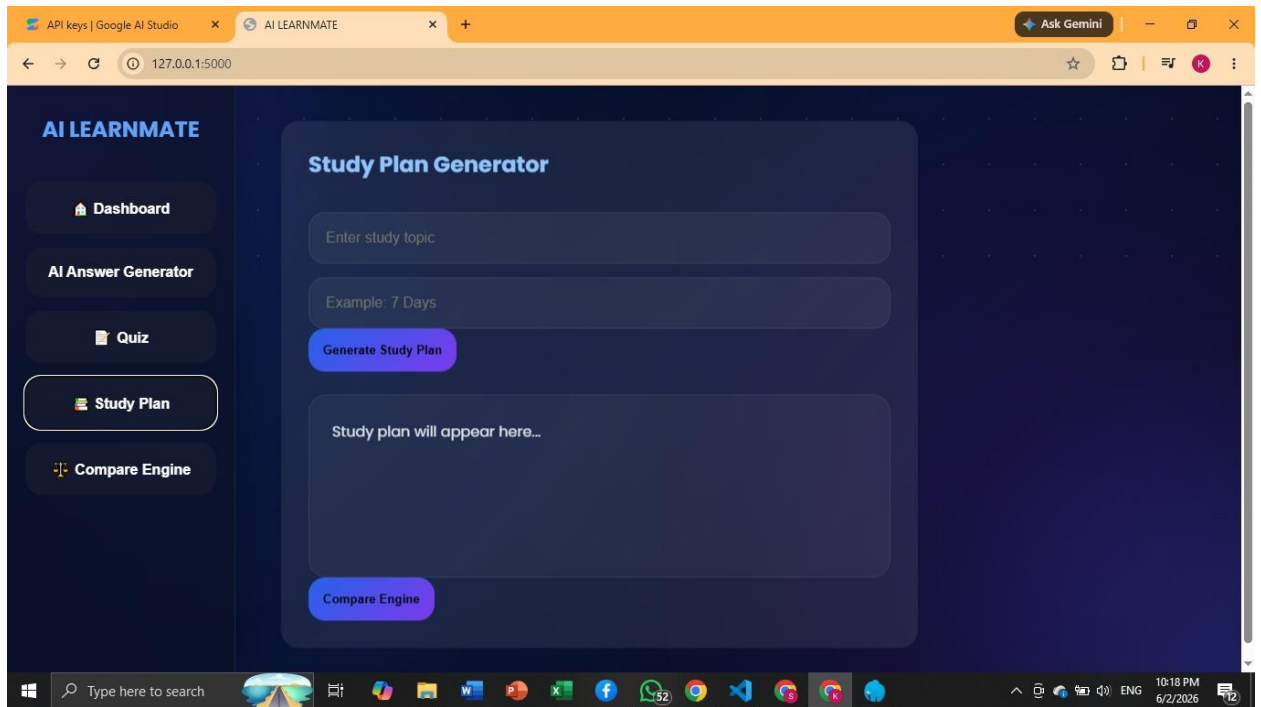
Beginner

Generate Answer

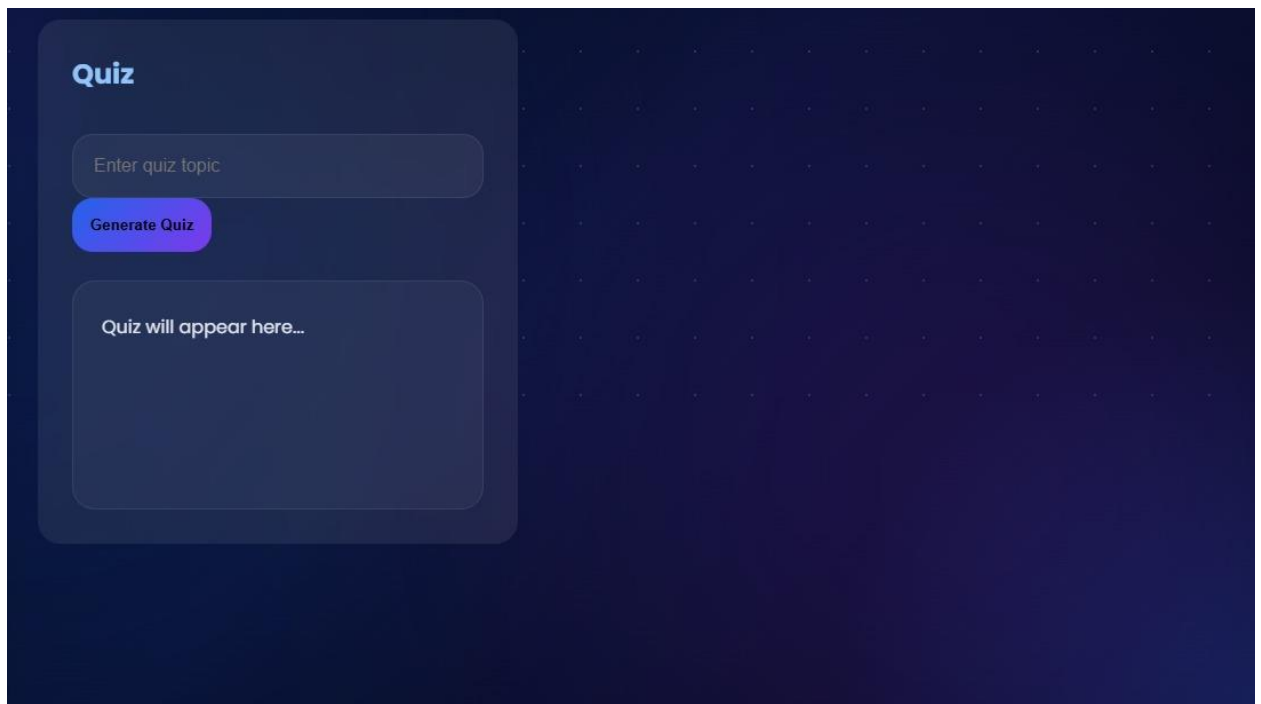
AI response will appear here...

The image shows a user interface for a 'Study Plan Generator'. It features a dark blue background. At the top is a large, rounded rectangular input field with the placeholder text 'Ask anything...'. Below this is a horizontal bar containing a dropdown menu currently set to 'Beginner', with a small downward arrow on the right. Underneath the dropdown is a bright blue, rounded rectangular button with the text 'Generate Answer' in white. At the bottom is another large, rounded rectangular area intended for the AI's response, with the placeholder text 'AI response will appear here...'.

- Study Plan Generator



- Quiz Generator



- Compare Engine

